

CMPG 311 PROJECT

PHASE 3

PAWS AND CLAWS RESCUE

Submission date: 08/05/2026

Group Members:

50143956 - Christoph van Jaarsveld - (Group Leader)

46251960 - Luan du Plessis

49019767 - Pieta Hattingh

45926808 - Sune Muller

46036482 - Zoe du Plessis



B.Sc. Information Technology
NORTH-WEST UNIVERSITY - DISTANCE

Contents

Phase 1	3
1. Introduce members of the group	Error! Bookmark not defined.
Names, surnames and student numbers	Error! Bookmark not defined.
Contact information	Error! Bookmark not defined.
Photos	Error! Bookmark not defined.
Indicate group leader	3
2. Analyse a company situation	3
Company objectives (mission of the environment)	3
Company operations (general operating environment)	3
Company organizational structure	4
3. Define problems and constraints	5
4. Database system specifications	5
Define objectives to solve problems identified	5
Information the company requires from the database	6
Scope	6
Boundaries	7
Phase 2	8
1. Introduction	8
2. Business Rules	8
3. Entity Relationship diagram:	10
4. Notes on the ER diagram:	10
5. Logical Data Model	13
5.1. Mapping the Conceptual Model to the Logical Model	13
5.2. Logical Model	13
5.3. Dependency Diagrams	14
Phase 3	15
Database objects	15
1.1 Tables	16
1.2 Indexes	26
1.3 Views	27
1.4 Data loading	28
Queries	36
2.1 Limitation of rows and columns	36
2.2 Sorting	37
2.3 LIKE, AND & OR	37
2.4 Character functions	38

2.5 Round	39
2.6 Variables	39
2.7 Trunc.....	40
2.8 Date	40
2.9 Date Function.....	41
2.10 Aggregates	41
2.11 Group By	42
2.12 Having.....	42
2.13 Join (Inner).....	43
2.14 Join (Multiple)	44
2.15 Sub-query	45
2.16 Sub-query	45
2.17 Outer Join.....	46
2.18 Math & Logic.....	46
Extra Functionality	47
2.19 High Value Donors support Critical Animals.....	47
2.20 Medical Provision Utilisation Ratio	47
2.21 Complete Animal History	48

Phase 1

Indicate group leader

Christoph is our group leader.

2. Analyse a company situation

Company objectives (mission of the environment)

Paws and Claws Rescue is a company aimed at rescuing stray animals, giving them the necessary medical treatment and finding their loving forever-homes. This company needs a system to manage all the information about the animals in their care. The information that will need to be managed includes medical records for the animals, how the animals were rescued and their adoption status. Some other useful functions that the system may have are storing information about people that want to adopt and medical treatments.

By having a system, it will help Paws and Claws Rescue to keep accurate records that are up to date. It will also make it easier for the company to track animals and their data.

Company operations (general operating environment)

Flow of operations of Paws and Claws Rescue:

When an abandoned, injured, or surrendered animal is brought to the rescue clinic, the administrative staff registers the animal and records basic details, such as the animal type, breed, age, condition, and date of arrival. The animal is then assigned to an available kennel in the shelter.

A veterinarian examines the animal and assesses the condition of the animal. This is done to identify the health condition of the animal. Veterinary assistants assist in the administration of any treatment or vaccination required by the animal.

The volunteers assist the animal caretaker staff at daily tasks such as feeding the animals, cleaning the kennels, and spending time with the animals so that they are better prepared for adoption.

During the animal's stay at the shelter, its health condition is monitored and recorded continuously. Veterinarians update treatment records if additional medical care is needed, ensuring that the medical history of each animal is well documented.

When the animal is healthy enough and ready for adoption, it is listed as available for adoption. Potential adopters can apply to adopt the animal, and the application will then be processed by the adoption coordinator.

If the adoption application is successful, the process is completed and the details of the adopter and adoption fees are recorded.

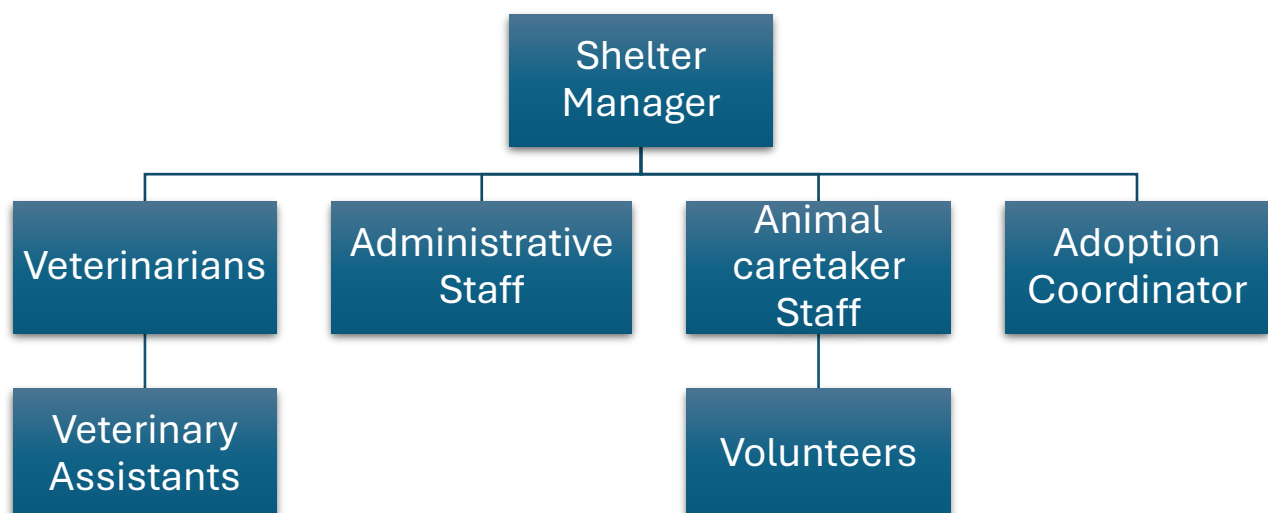
After adoption, the animal is removed from the Paws and Claws Rescue, and the kennel is made available for another rescued animal.

The shelter manager is in charge of all the daily activities of the shelter and ensures all the animals are well taken care of and that the kennels are not overcrowded. The manager also depends on the information provided by the database system for monitoring shelter activities.

Business Rules

- Each animal entering the shelter must be added to the system.
- Each animal must be assigned to one kennel at a time.
- Each kennel can hold one animal at a time.
- Each animal must undergo a medical examination after entering the shelter.
- Each veterinarian can treat multiple animals.
- Each animal can receive multiple treatments
- Only qualified veterinarians are permitted to record medical examinations and treatments for the animals.
- An animal can only be marked as adoptable after it has been medically cleared by a veterinarian.
- Each adoption is associated with one registered adopter.
- An adopter can adopt more than one animal, and each animal can only be adopted once.
- Adoption fees are recorded when an adoption is performed.
- When an animal is adopted, its kennel becomes available for another animal.
- Each donation is made by one donor.
- A donor can make multiple donations.

Company organizational structure



3. Define problems and constraints

Problems in the current environment

1. Manual and/or scattered records cause gaps in medical history, intake notes and treatment outcomes. This has a negative effect on animal welfare decisions.
2. Due to the lack of a central record of kennel assignments and availability, the shelter struggles to manage capacity. This leads to overcrowding or inefficient usage of space.
3. Adoption applications and paperwork can be delayed or lost, slowing down the rehoming process.
4. Management cannot make reliable summaries and decisions when reporting is weak.
5. Supplies (Medicine and food) are hard to monitor using manual methods, this leads to shortages or overspending on unnecessary items.
6. Donations may be recorded inconsistently, making it hard to track donor history, totals per period and financial reporting needs.

Constraints

1. The solution must be cost-effective and realistic for a rescue organization with little or limited resources.
2. Databases must be designed and maintained using Oracle SQL developer.
3. The Adopter information must be protected (such as: Contact details, addresses etc.) requiring controlled access and careful handling of sensitive records.
4. The system must be user friendly and maintainable, since the organization may not have dedicated staff available.
5. The database must support simple processes and consistent data capturing even when staff/volunteers change regularly.

4. Database system specifications

Define objectives to solve problems identified

In order to address the problems and constraints above, the objective of this project is to design and implement a reliable operational database that supports daily processes. The database must improve record accuracy whilst simultaneously reducing administrative delays and provide consistent and accurate reports.

Objectives of the proposed database

1. Centralize all animal records from initial intake to final outcome whether rehomed, transferred, or fostered to establish a definitive single source of truth for every animal in the system.

2. Catalog medical interventions and associated costs include examinations, registrations, and sterilizations to provide a clear audit trail for clinical history and resource allocation.
3. Monitor real-time kennel occupancy and housing history to streamline space management, improve capacity planning, and mitigate the risks of overcrowding.
4. Facilitate the adoption pipeline by digitizing applications and approval workflows which minimizes clerical errors and accelerates the rehoming process.
5. Log staff and volunteer activity to enhance operational oversight, simplify scheduling, and ensure clear accountability across all daily shifts.
6. Maintain a detailed ledger of donations and donor history to improve financial transparency and provide a data driven foundation for future fundraising strategies.
7. Track inventory levels for essential supplies like food and medicine utilizing reorder triggers to prevent critical shortages and minimize unnecessary waste.

Quality and control objectives

Data integrity: Prevent duplicates and inconsistent records by using primary keys, foreign keys, and appropriate constraints.

Access control: Support different user roles (e.g., admin vs general staff) to protect sensitive adopter data and reduce accidental edits.

Efficient reporting: Structure data so that key operational and management reports can be generated quickly (e.g., monthly adoptions, animals under treatment, low stock items, donation totals).

Information the company requires from the database

The shelter requires the following information from the database:

- A list of animals currently available for adoption.
- Animals under medical treatment.
- Kennel occupancy report.
- Adoption history of the animals.
- The number of animals adopted per month.
- Veterinary treatment history for each animal.

Scope

The system will store and manage data about the animals that Paws and Claws rescues. The information that the system should include about the animals is their breed, age, medical history, rescue date and adoption status. Some information that the system may also include is data about people that want to adopt and animals that are successfully adopted.

The system will also store information about staff members and medical treatments. For medical treatments the system will store information like vaccinations and check-ups to ensure the animals get all the necessary treatments.

By doing the above-mentioned the system will help the company keep track of the animals, prevent errors like outdated data and make day-to-day tasks easier and more efficient. There will be an administrator that can manage the system. The everyday staff will be able to view the database and add records.

Boundaries

The following boundaries are applicable to our database:

- Finances – We will need to ensure that financial limits do not slow or stop our database development. This means we will need to work closely with the company to ensure a fair budget.
- Time – We need to make sure we get everything done in time so that the company can start using their system as soon as possible.
- Human Resources – A smaller team is working on the database; this may slow down development if we do not plan ahead.
- Software and Hardware – The company uses different kinds of computers, we will need to ensure that the database software is able to run on all of the computers, otherwise this could also cause a problem.

All of the above-mentioned boundaries will need to be taken into consideration and close monitoring. This will ensure that the system will still be of high quality and that it will be done on time.

Phase 2

1. Introduction

In this assignment, we will be focusing on the database design stage of the database life cycle. The business operations discussed in the first section will be converted into business rules for this section. A conceptual ER diagram will be created to demonstrate the entities, attributes and relationships that will be present in the new database as well as a logical model will also be created of the business environment. These will form the basis of our database, and it is crucial that these are created as accurately and correctly as possible.

2. Business Rules

The following day to day operations take place at Paws and Claws Rescue:

1. Animal intake and shelter placement
2. Medical examination and treatment
3. Adoption processing
4. Donations
5. Shelter and staff management

Business rules on animal intake and shelter placement

- Each animal that enters the shelter must be registered on the system.
- Each animal must be assigned to one kennel at a time.
- Each kennel can only hold one animal at a time.
- Once an animal is adopted, its kennel becomes available for a new animal.

Business rules on medical examination and treatment

- Each animal must undergo a medical examination after entering the shelter.
- One veterinarian can help more than one animal.
- Each animal can get more than one treatment.
- Only veterinarians who are qualified are allowed to record medical examinations and perform treatments.
- A veterinarian must medically clear an animal before it can be marked as adoptable.

Business rules on adoption processing

- There is only one registered adopter for each adoption.
- An adopter can adopt more than one animal.
- Each animal can only be adopted once.
- When an adoption is complete, adoption fees must be recorded.

- Once an animal is adopted it is removed from the shelter system.

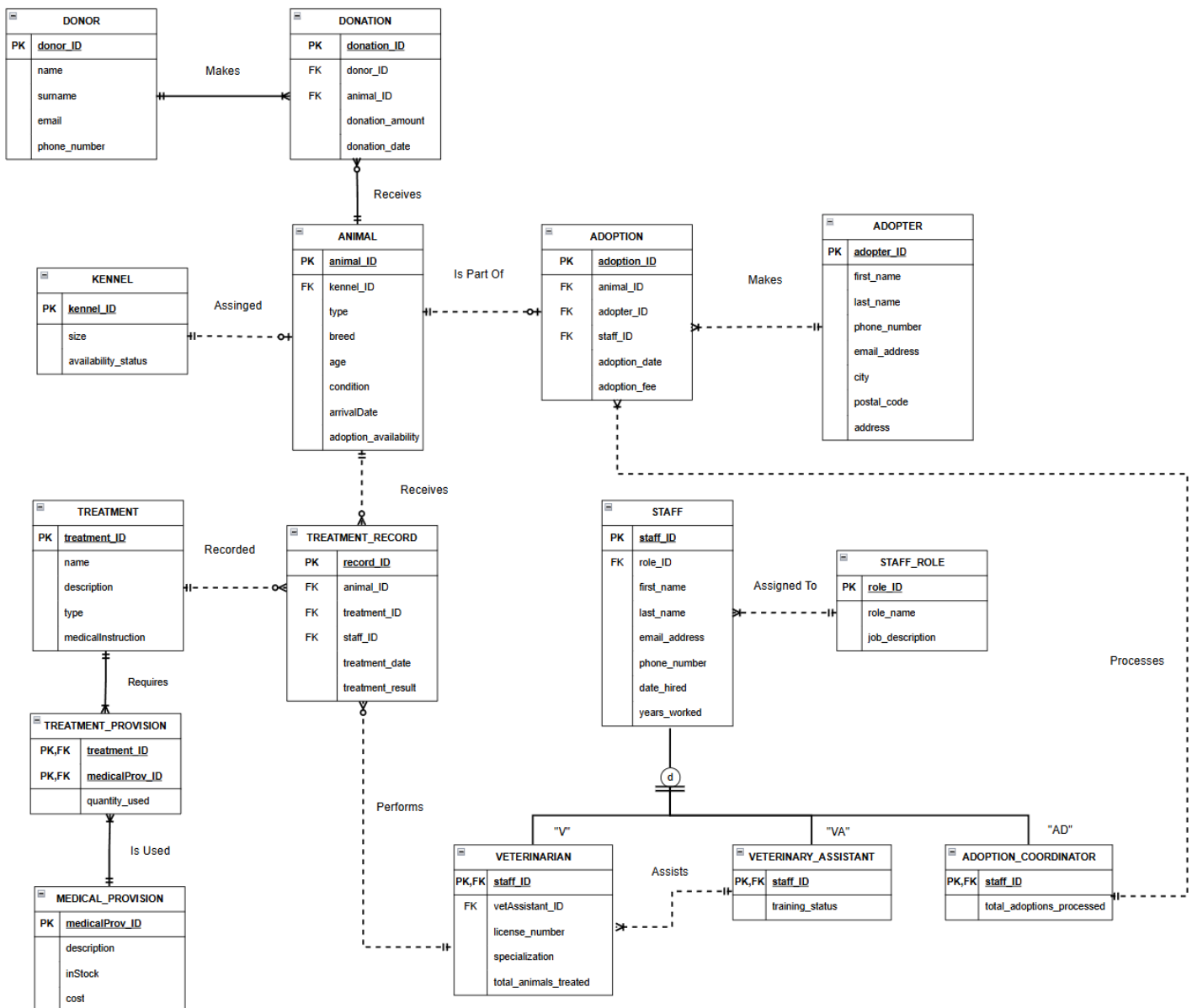
Business rules on donations

- Each donation is made by one donor.
- A donor can make many donations.

Business rules on shelter and staff management

- The shelter manager is in charge of what happens at the shelter every day.
- The shelter manager uses the database to monitor shelter operations .
- Volunteers help with everyday chores such as feeding, cleaning kennels and spending time with animals.

3. Entity Relationship diagram:



4. Notes on the ER diagram:

1. Composite / Bridge Entities

We use bridge entities to resolve many-to-many relationships.

For example: DONATION is a bridge entity between DONOR and ANIMAL, because a donor can donate to many animals and an animal can receive donations from many donors, so DONATION helps to solve this M:M relationship. The same can be said for TREATMENT_RECORD, it records which staff performed which treatment on which animal. It is a bridge entity between ANIMAL, TREATMENT and STAFF.

2. Composite Keys

We used composite keys in:

- TREATMENT_PROVISION

- Primary Key = (*treatment_ID* + *medicalProv_ID*)
- Ensures each medication used per treatment is uniquely recorded

3. Weak vs Strong Entities

Our Strong Entities are:

- DONOR
- ANIMAL
- ADOPTER
- STAFF
- KENNEL
- TREATMENT

Our Weak Entities are:

- DONATION
 - Depends on DONOR and ANIMAL
- TREATMENT_PROVISION
 - Depends on TREATMENT and MEDICAL_PROVISION

4. Optional vs Mandatory Relationships

Mandatory Relationships

- ANIMAL → KENNEL
 - Every animal must be assigned to a kennel
- ADOPTION → ANIMAL
 - Every adoption must involve an animal
- DONATION → DONOR
 - Every donation must have a donor

Optional Relationships

- ANIMAL → ADOPTION
 - An animal is not always adopted immediately
- ANIMAL → TREATMENT_RECORD
 - An animal may not require treatment

5. One-to-Many (Cardinality & Participation)

Examples:

- DONOR → DONATION
 - One donor can make many donations
 - Each donation belongs to one donor
- ADOPTER → ADOPTION
 - One adopter can adopt many animals
 - Each adoption has only one adopter
- KENNEL → ANIMAL
 - One kennel holds one animal at a time
 - Over time, it can hold many animals

6. Supertype / Subtype (Generalization)

You also have a specialization hierarchy:

- Supertype: STAFF
- Subtypes:
 - VETERINARIAN
 - VETERINARY_ASSISTANT
 - ADOPTION_COORDINATOR

This shows:

- Shared attributes (name, email, etc.) in STAFF
- Specialized attributes (e.g., license_number) in subtypes

7. Derived Attributes

- years_worked (STAFF)
 - Can be derived from hire date
- total_animals_treated (VETERINARIAN)
 - Can be calculated from TREATMENT_RECORD

5. Logical Data Model

5.1. Mapping the Conceptual Model to the Logical Model

- Each strong entity in the conceptual model was mapped to its own relation in the logical model, for example DONOR, ANIMAL, ADOPTER, KENNEL, STAFF, TREATMENT and MEDICAL_PROVISION.
- One-to-many relationships were implemented by placing the primary key of the parent entity as a foreign key in the child entity. For example, kennel_ID was included in ANIMAL, role_ID was included in STAFF, and animal_ID, adopter_ID and staff_ID were included in ADOPTION.
- Many-to-many relationships were resolved by creating bridge entities. DONATION was used to resolve the relationship between DONOR and ANIMAL, while TREATMENT_PROVISION was used to resolve the relationship between TREATMENT and MEDICAL_PROVISION.
- Composite keys were used in bridge entities where necessary. TREATMENT_PROVISION uses the composite key (treatment_ID, medicalProv_ID).
- The supertype/subtype structure in the conceptual model was mapped by keeping STAFF as the supertype and creating separate subtype relations for VETERINARIAN, VETERINARY_ASSISTANT and ADOPTION_COORDINATOR. The primary key of STAFF was reused in each subtype.
- Derived attributes were identified in the conceptual model, such as years_worked and total_animals_treated. These can be calculated from existing data, although they are still shown in the logical model for business reporting purposes.

5.2. Logical Model

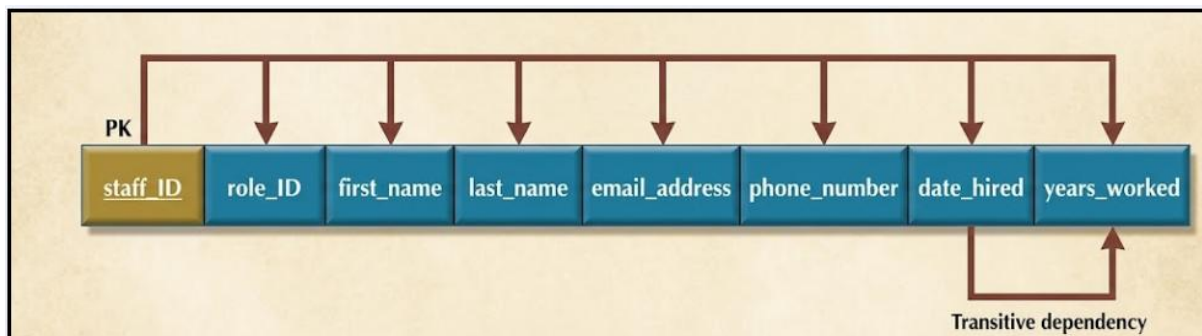
- **DONOR** (donor_ID, name, surname, email, phone_number)
- **DONATION** (donation_ID, *donor_ID*, *animal_ID*, donation_amount, donation_date)
- **KENNEL** (kennel_ID, size, availability_status)
- **ANIMAL** (animal_ID, *kennel_ID*, type, breed, age, condition, arrivalDate, adoption_availability)
- **ADOPTER** (adopter_ID, first_name, last_name, phone_number, email_address, city, postal_code, address)
- **ADOPTION** (adoption_ID, *animal_ID*, *adopter_ID*, *staff_ID*, adoption_date, adoption_fee)
- **TREATMENT** (treatment_ID, name, description, type, prescribed_medication)
- **MEDICAL_PROVISION** (medicalProv_ID, description, inStock, cost)
- **TREATMENT_PROVISION** (*treatment_ID*, *medicalProv_ID*, quantity_used)
- **STAFF_ROLE** (role_ID, role_name, job_description)
- **STAFF** (staff_ID, *role_ID*, first_name, last_name, email_address, phone_number, date_hired, years_worked)
- **VETERINARY_ASSISTANT** (*staff_ID*, training_status)
- **VETERINARIAN** (*staff_ID*, *vetAssistant_ID*, license_number, specialization, total_animals_treated)
- **ADOPTION_COORDINATOR** (*staff_ID*, total_adoptions_processed)

- **TREATMENT_RECORD** (record_ID, animal_ID, treatment_ID, staff_ID, treatment_date, treatment_result)

5.3. Dependency Diagrams

- Dependency diagrams were used to test 1NF, 2NF and 3NF.
- They show whether non key attributes depend on the key, the whole key, and nothing but the key.

STAFF

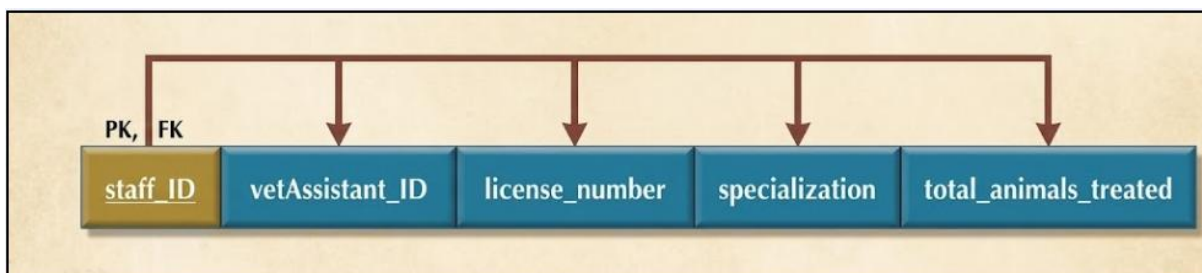


1NF (staff_ID, role_ID, first_name, last_name, email_address, phone_number, date_hired, years_worked)

PARTIAL DEPENDENCIES: (None)

TRANSITIVE DEPENDENCY: (date_hired -> years_worked)

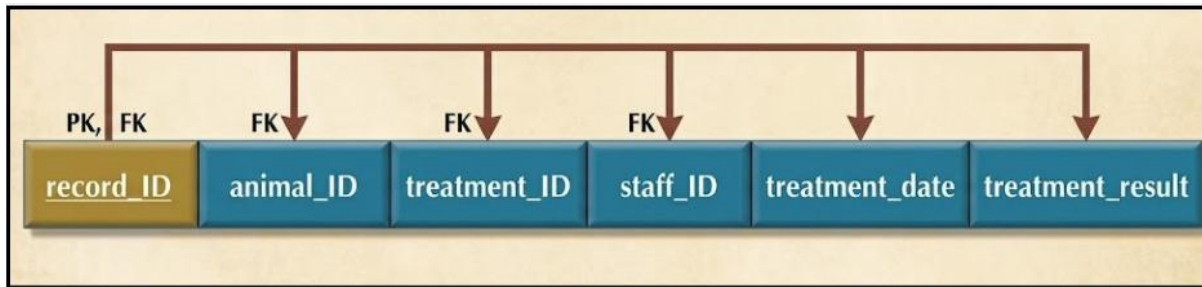
VETERINARIAN



1NF (staff_ID, vetAssistant_ID, license_number, specialization, total_animals_treated)

PARTIAL DEPENDENCIES: (None)

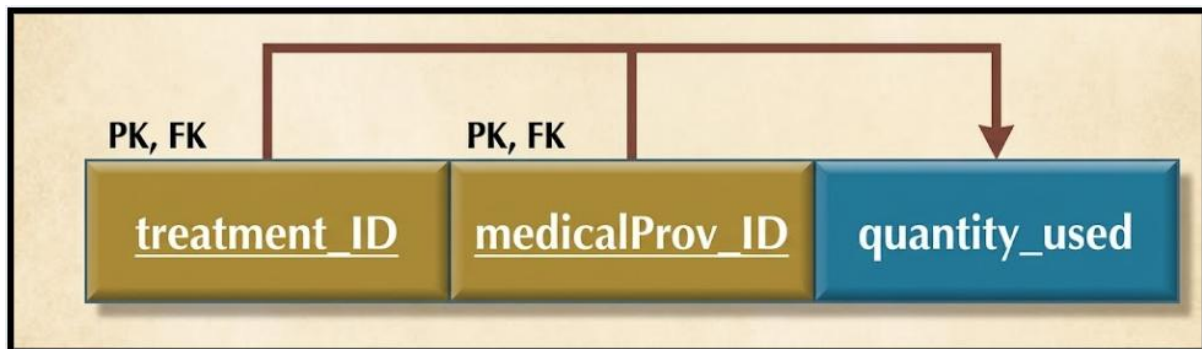
TRANSITIVE DEPENDENCY: (None)

TREATMENT_RECORD

1NF: (record_ID, animal_ID, treatment_ID, staff_ID, treatment_date, treatment_result)

PARTIAL DEPENDENCIES: (None)

TRANSITIVE DEPENDENCY: (None)

TREATMENT_PROVISION

1NF (treatment_ID, medicalProv_ID, quantity_used)

PARTIAL DEPENDENCIES: (None)

TRANSITIVE DEPENDENCY: (None)

Phase 3

Database objects

In this section the initial database is created including all tables, relationships, views, indexes and sequences. Once the database is created, it is populated with default data entries. This data is used to demonstrate queries that meet the business requirements of Paws and Claws Rescue. In order to ensure a clean setup any existing tables and sequences are dropped beforehand, preventing duplication and avoiding conflicts with previously created database objects

```
=====
-- DROP EXISTING TABLES (clean start)
--
```

```
=====
BEGIN
```

```
  FOR t IN (
```



```

SELECT table_name FROM user_tables
WHERE table_name IN (
    'DONATION', 'TREATMENT_RECORD', 'TREATMENT_PROVISION',
    'ADOPTION', 'VETERINARIAN', 'VETERINARY_ASSISTANT',
    'ADOPTION_COORDINATOR', 'ANIMAL', 'ADOPTER', 'STAFF',
    'TREATMENT', 'MEDICAL_PROVISION', 'KENNEL', 'DONOR', 'STAFF_ROLE'
)
) LOOP
    EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE
CONSTRAINTS';
END LOOP;
END;
/

=====
-- DROP EXISTING SEQUENCES (clean start)
--
=====
BEGIN
    FOR s IN (
        SELECT sequence_name FROM user_sequences
        WHERE sequence_name IN (
            'ANIMAL_SEQ', 'KENNEL_ID_SEQ', 'DONOR_ID_SEQ', 'ADOPTER_ID_SEQ',
            'ADOPTION_ID_SEQ', 'STAFF_ID_SEQ', 'STAFF_ROLE_ID_SEQ',
            'TREATMENTID_SEQ', 'MEDICALPROV_ID_SEQ', 'TREATMENTRECORD_SEQ',
            'DONATION_ID_SEQ'
        )
    ) LOOP
        EXECUTE IMMEDIATE 'DROP SEQUENCE ' || s.sequence_name;
    END LOOP;
END;
/

```

1.1 Tables

ADOPTER Table

The ADOPTER table was created to store the personal and contact details of individuals who adopt animals from Paws and Claws Rescue. Each adopter is assigned a unique identifier, adopter_ID, which serves as the primary key. The table also includes attributes such as first name, last name, phone number, email address, city, postal code, and physical address.

Textual fields are stored using the VARCHAR2 data type, while the adopter_ID uses the NUMBER data type. A UNIQUE constraint is applied to the email_address field to ensure that no two adopters are registered with the same email.

The table is in third normal form (3NF), as all non-key attributes depend solely on the primary key, and there are no partial or transitive dependencies.

SQL statement:

```

-- ADOPTER: People who adopt animals
-- -----
CREATE TABLE ADOPTER (
    adopter_ID NUMBER(5) NOT NULL,
    first_name VARCHAR2(50) NOT NULL,
    last_name VARCHAR2(50) NOT NULL,
    phone_number VARCHAR2(15),
    email_address VARCHAR2(100) UNIQUE,

```

```

    city VARCHAR2(50),
    postal_code VARCHAR2(10),
    address VARCHAR2(150),
    CONSTRAINT Adopter_PK PRIMARY KEY (Adopter_ID)
);

```

A sequence, Adopter_ID_seq, is used to automatically generate unique adopter IDs. It starts at 1 and increments by 1 up to a maximum of 100,000, ensuring that each adopter is assigned a unique identifier without manual input.

SQL statement:

```

-- Create SEQUENCE for ADOPTER table
CREATE SEQUENCE Adopter_ID_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 100000
NOCYCLE;

```

ADOPTION Table

The ADOPTION table records all completed animal adoptions within the rescue organization. Each adoption is uniquely identified by adoption_ID, which serves as the primary key. The table also stores references to the adopted animal, the adopter, and the staff member who facilitated the adoption, along with the adoption date and adoption fee.

This table includes three foreign keys:

adopter_ID, which references the ADOPTER table

staff_ID, which references the ADOPTION_COORDINATOR table

animal_ID, which references the ANIMAL table

These relationships ensure referential integrity across the database. The adoption_fee field includes a CHECK constraint to ensure that only non-negative values are recorded.

Numeric fields use the NUMBER data type, while the adoption_date uses the DATE data type. The table is in third normal form (3NF), as all attributes are fully dependent on the primary key and no transitive dependencies exist.

SQL statement:

```

-- ADOPTION: Records of completed adoptions
-- -----
CREATE TABLE ADOPTION (
    adoption_ID NUMBER(5) NOT NULL,
    animal_ID NUMBER(3) NOT NULL,
    adopter_ID NUMBER(5) NOT NULL,
    staff_ID NUMBER(4) NOT NULL,
    adoption_date DATE NOT NULL,
    adoption_fee NUMBER(10,2) CHECK (adoption_fee >= 0) NOT NULL,

    CONSTRAINT Adoption_PK PRIMARY KEY (Adoption_ID),
    CONSTRAINT FK_Adopter FOREIGN KEY (Adopter_ID)
        REFERENCES ADOPTER(Adopter_ID),
    CONSTRAINT FK_Coordinator FOREIGN KEY (staff_ID)
        REFERENCES ADOPTION_COORDINATOR(staff_ID),
    CONSTRAINT FK_Adoption_animal FOREIGN KEY (animal_ID)

```

```

REFERENCES ANIMAL(animal_ID)
);

```

A sequence, Adoption_ID_seq, is used to automatically generate unique adoption IDs, starting from 1 and incrementing by 1 up to 100,000.

SQL statement:

```

-- Create SEQUENCE for ADOPTION table
CREATE SEQUENCE Adoption_ID_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 100000
NOCYCLE;

```

ADOPTION_COORDINATOR Table

The ADOPTION_COORDINATOR table represents a specialized sub-entity of the STAFF table, specifically for staff members responsible for processing animal adoptions. Each coordinator is identified by staff_ID, which serves as both the primary key and a foreign key referencing the STAFF table.

The table includes an additional attribute, total_adoptions_processed, which tracks the number of adoptions handled by each coordinator. This field has a default value of 0 and includes a CHECK constraint to ensure that only non-negative values are recorded.

Since this table depends on the STAFF table for its primary key, it reinforces an inheritance relationship between general staff members and adoption coordinators.

The table is in third normal form (3NF), as all attributes depend entirely on the primary key, with no redundancy or transitive dependencies present.

SQL statement:

```

-- ADOPTION_COORDINATOR: Sub-entity of STAFF
-- -----
CREATE TABLE ADOPTION_COORDINATOR (
    staff_ID NUMBER(4),
    total_adoptions_processed NUMBER(4) DEFAULT 0 CHECK
(total_adoptions_processed >= 0) NOT NULL,
    CONSTRAINT Coordinator_PK PRIMARY KEY (staff_ID),
    CONSTRAINT fk_adoptcoord_staff FOREIGN KEY (staff_ID) REFERENCES
STAFF(staff_ID)
);

```

ANIMAL Table

The ANIMAL table was created to store all rescued animals that are admitted into Paws and Claws Rescue. Each animal has a unique ID number assigned individually, animal_ID, which functions as the primary key. Other attributes are also included: kennel allocation, animal type, breed, age, physical condition, date of arrival and adoption availability. Numerical fields use the NUMBER data type, the textual descriptions use VARCHAR2, the arrival date uses the DATE data type. The table is in third normal form, because all non-key attributes depend on the primary key and no transitive dependencies are present.

SQL statement:

```

-- ANIMAL: Animals currently or previously in the rescue

```

```

-----
CREATE TABLE ANIMAL (
  animal_ID NUMBER(3) CONSTRAINT animal_id_pk PRIMARY KEY,
  kennel_ID NUMBER(3) NOT NULL,
  category VARCHAR2(30) NOT NULL,
  breed VARCHAR2(40) NOT NULL,
  age NUMBER(2) CHECK (age > 0) NOT NULL,
  condition VARCHAR2(50) NOT NULL,
  arrivalDate DATE NOT NULL,
  adoption_availability VARCHAR2(20) NOT NULL,
  CONSTRAINT adopt_avail CHECK (adoption_availability IN ('Available', 'Not
  Available')),
  CONSTRAINT fk_animal_kennel FOREIGN KEY (kennel_ID) REFERENCES
  KENNEL(kennel_ID)
);

```

DONATION Table

The DONATION table serves as a vital bridge between our donors and the animals they support. It captures the details of every gift, including who gave it, which animal it was for, the amount, and the date. It relies on two foreign keys: `fk_donation_donor`, which points to the DONOR table, and `fk_donation_animal`, which points to the ANIMAL table. To keep our records clean, a CHECK constraint ensures that all donation amounts are positive, while the `donation_date` defaults to the current system date if it is not manually entered.

There is no need for a separate sequence here, as the DONATION table builds its primary key directly from the IDs existing in the DONOR and ANIMAL tables.

SQL statement:

```

-- DONATION: Donations made by donors toward specific animals
-----

```

```

CREATE TABLE DONATION (
  donation_ID NUMBER CONSTRAINT pk_donation PRIMARY KEY,
  donor_ID NUMBER NOT NULL,
  animal_ID NUMBER NOT NULL,
  donation_amount NUMBER(10,2) NOT NULL,
  donation_date DATE DEFAULT SYSDATE,
  CONSTRAINT fk_donation_donor FOREIGN KEY (donor_ID)
  REFERENCES DONOR(donor_ID),
  CONSTRAINT fk_donation_animal FOREIGN KEY (animal_ID)
  REFERENCES ANIMAL(animal_ID),
  CONSTRAINT chk_donation_amount
  CHECK (donation_amount > 0)
);

```

DONOR Table

The DONOR table is dedicated to the contact details of the individuals and organizations supporting Paws and Claws Rescue. Each donor is assigned a unique `donor_ID` primary key, along with their name, surname, email, and phone number. We have a UNIQUE constraint on the email field to ensure no two donors are logged under the same address. Additionally, the `phone_number` is stored as text (VARCHAR2) rather than a number to properly preserve formatting like leading zeros or international plus signs. This table does not require any foreign keys.

SQL statement:

```
-- DONOR: People who make financial donations
-- -----
CREATE TABLE DONOR (
  donor_ID NUMBER,
  name VARCHAR2(50) NOT NULL,
  surname VARCHAR2(50) NOT NULL,
  email VARCHAR2(100) UNIQUE,
  phone_number VARCHAR2(10),
  CONSTRAINT pk_donor PRIMARY KEY (donor_ID)
);
```

We also use a sequence for the donor_ID to make sure every new supporter is automatically assigned a unique ID. It starts at 1 and increments up to 1,000, cycling back once it hits the limit. This saves the user from having to manually figure out what the next ID should be.

SQL statement:

```
-- CREATE SEQUENCE for DONOR table
CREATE SEQUENCE donor_ID_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 1000000
NOCYCLE;
```

KENNEL Table

The KENNEL table is where we keep track of every kennel at the shelter. Each entry includes a unique kennel_ID as its primary key, the size of the unit (Small, Medium, or Large), and its current availability_status—whether it is Available, Occupied, or closed for Maintenance. We’ve also included two CHECK constraints to safeguard the data, ensuring that only approved sizes and statuses can be recorded. This table stands on its own and does not use any foreign keys.

SQL statement:

```
-- KENNEL: Stores kennel/enclosure details
-- -----
CREATE TABLE KENNEL (
  kennel_ID NUMBER,
  sizes VARCHAR2(20) NOT NULL,
  availability_status VARCHAR2(20) NOT NULL,
  CONSTRAINT pk_kennel PRIMARY KEY (kennel_ID),
  CONSTRAINT chk_kennel_size
    CHECK (sizes IN ('Small', 'Medium', 'Large')),
  CONSTRAINT chk_kennel_status
    CHECK (availability_status IN ('Available', 'Occupied',
'Maintenance'))
);
```

To go along with the KENNEL table, we use a sequence to handle the kennel_ID values automatically. This makes things much simpler for the user and prevents any accidental duplicates or invalid keys. The sequence starts at 1 and goes up to 1,000 before starting over again.

SQL statement:

```
-- CREATE SEQUENCE for KENNEL table
CREATE SEQUENCE kennel_ID_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 10000
NOCYCLE;
```

MEDICAL_PROVISION Table

All of the medical supplies and medications that are available at Paws and Claws Rescue shelter are stored in the MEDICAL_PROVISION table. It tracks the description, current stock level, and unit cost of each provision item. Each record is uniquely identified by a medicalProv_ID which is the primary key. The description field is VARCHAR2(255) to allow a long name or notes for each provision, while the instock field uses a simple NUMBER to represent the number of quantities on hand. The cost is NUMBER(10, 2) to provide an accurate currency representation with cents. This table has no foreign keys.

SQL statement:

```
-- MEDICAL_PROVISION: Medical supplies/stock
-- -----
CREATE TABLE MEDICAL_PROVISION (
    medicalProv_ID NUMBER CONSTRAINT pk_medical_provision PRIMARY KEY,
    description VARCHAR2(225),
    inStock NUMBER CHECK (inStock >= 0),
    cost NUMBER(10,2) CHECK (cost >= 0)
);
```

Alongside this table a sequence is also created to automatically generate unique values for medicalProv_ID removing the risk of a user accidentally entering a duplicate or invalid primary key. The sequence starts at 1 and increments by 1 until the max value of 1000 is reached.

SQL statement:

```
-- CREATE SEQUENCE for MEDICAL_PROVISION table
CREATE SEQUENCE medicalProv_ID_seq
start with 1
increment by 1
minvalue 1
maxvalue 100000
NOCYCLE;
```

STAFF Table

The staff table stores general information about all the staff members. It includes personal and employment details such as name, contact information and employment duration. The primary key is the staff_ID. The role_ID is a foreign key referencing STAFF_ROLE making sure that each staff member is linked to a role.

SQL statement:

```
CREATE TABLE STAFF (  
    staff_ID NUMBER(4) CONSTRAINT staff_id_pk PRIMARY KEY,  
    role_ID NUMBER NOT NULL,  
    first_name VARCHAR2(50) NOT NULL,  
    last_name VARCHAR2(50) NOT NULL,  
    email_address VARCHAR2(100) UNIQUE NOT NULL,  
    phone_number VARCHAR2(10),  
    date_hired DATE NOT NULL,  
    years_worked NUMBER(2) CHECK (years_worked >= 0),  
    CONSTRAINT fk_staff_role FOREIGN KEY (role_ID) REFERENCES  
STAFF_ROLE(role_ID)  
);
```

Sequence for automatic staff_ID generations

SQL statement:

```
-- CREATE SEQUENCE for STAFF  
CREATE SEQUENCE staff_ID_seq  
START WITH 1  
INCREMENT BY 1  
MINVALUE 1  
MAXVALUE 100000  
NOCYCLE;
```

STAFF_ROLE Table

The STAFF_ROLE table stores all the roles that a staff member can have at the shelter including the role names and a description of the responsibilities. Each role uniquely identifies, which is the primary key, by the role_ID in the table. The role_name field is defined as VARCHAR2(50) and job_description as VARCHAR2(250) to allow detailed descriptions. This table has no foreign keys and acts as a reference table for STAFF.

SQL statement:

```
-- STAFF_ROLE: Role definitions for staff members
-- -----
CREATE TABLE STAFF_ROLE (
    role_ID NUMBER CONSTRAINT staffrole_id_pk PRIMARY KEY,
    role_name VARCHAR2(50) NOT NULL,
    job_description VARCHAR2(255));
```

A sequence is created for automatically generates unique role_ID values.

SQL statement:

```
-- CREATE SEQUENCE for staff roles
CREATE SEQUENCE staff_role_id_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 100
NOCYCLE;
```

TREATMENT Table

The TREATMENT table holds records for all types of treatments that can be given to animals in the shelter. The stored information includes a name and description of the treatment, the type of treatment (e.g., surgical, medicinal) and medical instructions. Each treatment has a unique treatment_ID which is the primary key. The field name is VARCHAR(50). The field type is VARCHAR(50) and medicalInstruction is VARCHAR2(255) to hold a variety of medical instructions. The table does not have any foreign keys as it is a reference table for the TREATMENT_PROVISION table.

SQL statement:

```
-- TREATMENT: Types of treatments that can be performed
-- -----
CREATE TABLE TREATMENT (
    treatment_ID NUMBER CONSTRAINT pk_treatment PRIMARY KEY,
    name VARCHAR2(50) NOT NULL,
    description VARCHAR2(255),
    category VARCHAR2(50),
    medicalInstruction VARCHAR2(255)
);
```

The created sequence treatmentId_seq is used to automatically assign primary values, the sequence starts at 1 and increments by 1 until the maximum value of 10000 is reached, ensuring uniqueness and prevention of user input errors.

SQL statement:

```
-- CREATE SEQUENCE for TREATMENT Table
CREATE SEQUENCE treatmentID_seq
start with 1
increment by 1
minvalue 1
maxvalue 100000
NOCYCLE;
```

TREATMENT_PROVISION Table

The TREATMENT_PROVISION table serves as a bridging entity that links the TREATMENT and MEDICAL_PROVISION tables. The table records the amount of the specific medical provision consumed during a specific treatment. The table uses a composite primary key created by combining both the treatment_ID and medicalProv_ID. The table contains two foreign key constraints, fk_treatment references the TREATMENT table via treatment_ID and fk_medicalProv_ID references the MEDICAL_PROVISION table via medicalProv_ID. The quantity_used field records the quantity of units that are used of a specific medical provision per treatment.

No sequences are needed for this table since its primary key values are derived from the foreign keys of its two parent tables.

SQL statement:

```
-- TREATMENT_PROVISION: Links treatments to medical provisions used
-----
CREATE TABLE TREATMENT_PROVISION (
  treatment_ID NUMBER,
  medicalProv_ID NUMBER,
  quantity_used NUMBER CHECK (quantity_used > 0),
  CONSTRAINT pk_treatment_provision
    PRIMARY KEY (treatment_ID, medicalProv_ID),
  CONSTRAINT fk_tp_treatment
    FOREIGN KEY (treatment_ID) REFERENCES TREATMENT (treatment_ID),
  CONSTRAINT fk_tp_medicalprov
    FOREIGN KEY (medicalProv_ID) REFERENCES
MEDICAL_PROVISION (medicalProv_ID)
);
```

TREATMENT_RECORD Table

The table TREATMENT_RECORD is created to act as a transaction table that records every treatment event performed on rescued animals. Each treatment record has a unique ID, record_ID. Foreign key references were created to both the ANIMAL and VETERINARIAN tables to preserve referential integrity between the treatment procedure, the animal receiving the treatment and the veterinarian responsible for the procedure. The table is important for maintaining an organised medical history of all the animals within the Paws and Claws.

SQL statement:

```
-- TREATMENT_RECORD: Records of treatments performed on animals
-- -----
CREATE TABLE TREATMENT_RECORD (
    record_ID NUMBER(3) CONSTRAINT treatmentrecord_id_pk PRIMARY KEY,
    animal_ID NUMBER(3) NOT NULL REFERENCES ANIMAL(animal_ID),
    treatment_ID NUMBER(4) NOT NULL REFERENCES TREATMENT(treatment_ID),
    staff_ID NUMBER(4) NOT NULL REFERENCES VETERINARIAN(staff_ID),
    treatment_date DATE NOT NULL,
    treatment_result VARCHAR2(100) NOT NULL
);
```

SQL statement:

```
-- CREATE SEQUENCE for TREATMENT_RECORD table
CREATE SEQUENCE treatmentrecord_seq
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 100000
NOCYCLE;
```

VETERINARIAN Table

The table VETERINARIAN table was created. It stores all veterinarian staff members responsible for the medical examinations and the treatment procedures performed at the shelter. Each veterinarian has a unique Id, staff_ID. The table also stores veterinarian assistant references, professional license numbers, specialization areas and the total number of animals treated. The UNIQUE constraint was applied to the license_number field to ensure that duplicate professional licenses cannot be entered. A CHECK constraint was also implemented to ensure that the number of treated animals cannot be negative.

SQL statement:

```
-- VETERINARIAN: Sub-entity of STAFF
-- -----
CREATE TABLE VETERINARIAN (
    staff_ID NUMBER(4) CONSTRAINT veterinarian_staff_pk PRIMARY KEY,
    vetAssistant_ID NUMBER(4),
    license_number VARCHAR2(20) UNIQUE NOT NULL,
    specialization VARCHAR2(50) NOT NULL,
    total_animals_treated NUMBER(3) DEFAULT 0 CHECK (total_animals_treated >=
0) NOT NULL,
    CONSTRAINT fk_vet_staff FOREIGN KEY (staff_ID) REFERENCES STAFF(staff_ID),
    CONSTRAINT fk_vet_assistant FOREIGN KEY (vetAssistant_ID) REFERENCES
VETERINARY_ASSISTANT(staff_ID)
);
```

VETERINARY_ASSISTANT Table

The VETERINARY_ASSISTANT table is a subtype to STAFF and stores the additional information regarding the veterinary assistance. The staff_ID is both a primary key and a foreign key referencing STAFF. The training_status indicates whether the assistant is in training or already a trained professional.

SQL statement:

```
-- VETERINARY_ASSISTANT: Sub-entity of STAFF
-----
CREATE TABLE VETERINARY_ASSISTANT (
    staff_ID NUMBER(4) CONSTRAINT vetassist_id_pk PRIMARY KEY,
    training_status VARCHAR2(30) NOT NULL,
    CONSTRAINT chk_training_status CHECK (training_status IN ('Completed', 'In
Progress', 'Not Started')),
    CONSTRAINT fk_vetassist_staff FOREIGN KEY (staff_ID) REFERENCES
STAFF(staff_ID)
);
```

1.2 Indexes

In addition to the views and sequences created, we also created indexes to further improve the performance of our database. Unlike primary keys which are automatically indexed by Oracle SQL, non-primary keys columns must be manually indexed. We have identified three columns across Paws and Claws Rescue database that justifies indexing such, as the adopter's email address, the kennel's availability status, and the animal's arrival date. By indexing these columns, we can ensure that common shelter operations such as viewing kennel availability for a new arrived animal or looking up and adopter's details are performed as efficiently as possible.

Adopter Email (idx_adopter_email)

An Index is created on the email_address field in the ADOPTER table. Email addresses are commonly used to retrieve information about a particular adopter or to view whether an adopter already exists within the database. Indexing this field will speed up search operations, especially when the number of adopter increase.

SQL statement:

```
CREATE INDEX idx_adopter_email
ON ADOPTER(email_address);
```

Animal Arrival Date (idx_arrival_date)

An index is created on the arrivalDate field in the ANIMAL table. Arrival dates can be used at Paws and Claws Rescue for filter criteria, for instance when generating reports of animals admitted within a certain period or determining which animals have been at the shelter the longest. Indexing this arrival date column will speed up range-based queries in this field.

SQL statement:

```
CREATE INDEX idx_arrival_date
ON ANIMAL(arrivalDate);
```

Breed field

An index breed field of the ANIMAL table is created. Different animals have different breeds. This makes identification easier.

SQL statement:

```
CREATE INDEX idx_animal_breed
ON ANIMAL(breed);
```

Kennel Availability (idx_kennel_status)

An index is created on the availability_status field in the KENNEL table. This column will be queried frequently when assigning new incoming animals to available kennels. Having an index here ensures that filtering for open kennels will be efficient which is important for when new animals arrive and need to be placed quickly.

SQL statement:

```
CREATE INDEX idx_kennel_status
ON KENNEL(availability_status);
```

Staff ID field

An index is created, the staff_ID field from the TREATMENT_RECORD table.

SQL statement:

```
CREATE INDEX idx_treatment_staff
ON TREATMENT_RECORD(staff_ID);
```

1.3 Views

Adopter Summary

This view summarizes adopter activity. It helps identify frequent adopters.

SQL statement:

```
-- Summarises total adoptions per adopter
CREATE OR REPLACE VIEW ADOPTER_SUMMARY AS
SELECT
    ad.adopter_ID,
    ad.first_name,
    ad.last_name,
    COUNT(a.adoption_ID) AS "Total Adoptions"
FROM ADOPTER ad
LEFT JOIN ADOPTION a ON ad.adopter_ID = a.adopter_ID
GROUP BY ad.adopter_ID, ad.first_name, ad.last_name;
```

All Adoptions

This view combines data from Adoption, Adopter and Adoption_Coordinator tables to display the full Adoption view.

SQL statement:

```
-- Full adoption view which includes adopter name, coordinator name, date and fee
CREATE OR REPLACE VIEW ALL_ADOPTIONS AS
SELECT
    a.adoption_ID,
    ad.first_name || ' ' || ad.last_name AS "Adopter Name",
    s.first_name || ' ' || s.last_name AS "Coordinator Name",
    a.adoption_date,
    a.adoption_fee
FROM ADOPTION a
JOIN ADOPTER ad ON a.adopter_ID = ad.adopter_ID
JOIN ADOPTION_COORDINATOR ac ON a.staff_ID = ac.staff_ID
JOIN STAFF s ON ac.staff_ID = s.staff_ID;
```

Approved Adoptions

This view filters approved adoptions to be able to retrieve successful adoption records.

SQL statement:

```
-- Filter approved adoptions
CREATE OR REPLACE VIEW Approved_Adoptions AS
SELECT *
FROM Adoption;
```

Available Animals

The AVAILABLE_ANIMALS view displays all animals currently available for adoption.

SQL statement:

```
-- Lists all animals currently available for adoption
CREATE OR REPLACE VIEW AVAILABLE_ANIMALS AS
SELECT animal_ID, category, breed, age, condition
FROM ANIMAL
WHERE adoption_availability = 'Available';
```

Vet treatment summary

VET_TREATMENT_SUMMARY view combines veterinarian treatment information into a single report.

SQL statement:

```
-- Shows each vet alongside their treatment records
CREATE OR REPLACE VIEW VET_TREATMENT_SUMMARY AS
SELECT v.staff_ID, v.specialization, t.record_ID, t.treatment_result
FROM VETERINARIAN v
JOIN TREATMENT_RECORD t ON v.staff_ID = t.staff_ID;
```

1.4 Data loading

The following SQL statements populate the database tables with sample records. The data must be inserted after the tables have been created. Parent tables are inserted before child tables to prevent foreign-key errors.

ADOPTER

These insert statements load people who adopted or may adopt animals. Their contact details and address information support the adoption process and later follow-up communication.

SQL statement:

```
-- ADOPTER entries
-- -----
INSERT INTO ADOPTER (adopter_ID, first_name, last_name, phone_number,
email_address, city, postal_code, address)
VALUES (1, 'Carla', 'Jacobs', '0811111111', 'carla.jacobs@gmail.com',
'Springfield', '1449', '12 Rose Street');

INSERT INTO ADOPTER (adopter_ID, first_name, last_name, phone_number,
email_address, city, postal_code, address)
VALUES (2, 'Neo', 'Mabena', '0822222222', 'neo.mabena@yahoo.com',
'Potchefstroom', '2531', '45 Oak Avenue');

INSERT INTO ADOPTER (adopter_ID, first_name, last_name, phone_number,
email_address, city, postal_code, address)
VALUES (3, 'Sarah', 'Williams', '0833333333', 'sarah.williams@gmail.com',
'Springfield', '1450', '78 Lily Road');
```

ADOPTION table

These insert statements load completed adoption transactions. Each adoption links an animal, adopter and staff member, while also recording the adoption date and adoption fee.

SQL statement:

```
-- ADOPTION entries
-- -----
INSERT INTO ADOPTION (adoption_ID, animal_ID, adopter_ID, staff_ID,
adoption_date, adoption_fee)
VALUES (1, 1, 1, 401, TO_DATE('2026-03-21', 'YYYY-MM-DD'), 850.00);

INSERT INTO ADOPTION (adoption_ID, animal_ID, adopter_ID, staff_ID,
adoption_date, adoption_fee)
VALUES (2, 3, 2, 402, TO_DATE('2026-03-22', 'YYYY-MM-DD'), 550.00);

INSERT INTO ADOPTION (adoption_ID, animal_ID, adopter_ID, staff_ID,
adoption_date, adoption_fee)
VALUES (3, 4, 3, 401, TO_DATE('2026-03-23', 'YYYY-MM-DD'), 1250.00);
```

ADOPTION_COORDINATOR table

These insert statements load adoption coordinator subtype records. The total_adoptions_processed field is useful for management reports on adoption workload.

SQL statement:

```
-- ADOPTION_COORDINATOR entries
-- -----
INSERT INTO ADOPTION_COORDINATOR (staff_ID, total_adoptions_processed) VALUES
(401, 24);
INSERT INTO ADOPTION_COORDINATOR (staff_ID, total_adoptions_processed) VALUES
(402, 18);
```

ANIMAL table

These insert statements load animal records currently handled by Paws and Claws Rescue. Each animal is linked to a kennel and stores basic information such as type, breed, age, condition, arrival date and adoption availability.

SQL statement:

```
-- ANIMAL entries
-- -----
INSERT INTO ANIMAL (animal_ID, kennel_ID, category, breed, age, condition,
arrivalDate, adoption_availability)
VALUES (1, 101, 'Dog', 'Labrador', 3, 'Healthy', TO_DATE('2026-04-10', 'YYYY-
MM-DD'), 'Available');

INSERT INTO ANIMAL (animal_ID, kennel_ID, category, breed, age, condition,
arrivalDate, adoption_availability)
VALUES (2, 102, 'Cat', 'Siamese', 2, 'Minor Injury', TO_DATE('2026-04-18',
'YYYY-MM-DD'), 'Not Available');

INSERT INTO ANIMAL (animal_ID, kennel_ID, category, breed, age, condition,
arrivalDate, adoption_availability)
VALUES (3, 103, 'Dog', 'German Shepherd', 5, 'Recovering', TO_DATE('2026-03-
10', 'YYYY-MM-DD'), 'Available');

INSERT INTO ANIMAL (animal_ID, kennel_ID, category, breed, age, condition,
arrivalDate, adoption_availability)
VALUES (4, 104, 'Cat', 'Tabby', 1, 'Healthy', TO_DATE('2026-04-25', 'YYYY-MM-
DD'), 'Available');

INSERT INTO ANIMAL (animal_ID, kennel_ID, category, breed, age, condition,
arrivalDate, adoption_availability)
VALUES (5, 105, 'Dog', 'Border Collie', 4, 'Critical', TO_DATE('2026-04-28',
'YYYY-MM-DD'), 'Not Available');
```

DONATION table

These insert statements load donation transactions. Each donation links a donor to an animal and records the amount and date of the donation.

SQL statement:

```
-- DONATION entries (donation_ID added as surrogate PK)
-- -----
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (1, 1, 1, 750.00, TO_DATE('2026-04-11', 'YYYY-MM-
DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (2, 2, 1, 350.00, TO_DATE('2026-04-13', 'YYYY-MM-
DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (3, 3, 2, 1200.00, TO_DATE('2026-04-19', 'YYYY-MM-
DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (4, 4, 2, 250.00, TO_DATE('2026-04-21', 'YYYY-MM-
DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (5, 5, 5, 900.00, TO_DATE('2026-04-29', 'YYYY-MM-
DD'));
```

```

INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (6, 6, 5, 1100.00, TO_DATE('2026-04-30', 'YYYY-MM-DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (7, 1, 2, 500.00, TO_DATE('2026-04-22', 'YYYY-MM-DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (8, 2, 3, 450.00, TO_DATE('2026-03-15', 'YYYY-MM-DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (9, 3, 1, 300.00, TO_DATE('2026-04-14', 'YYYY-MM-DD'));
INSERT INTO DONATION (donation_ID, donor_ID, animal_ID, donation_amount,
donation_date) VALUES (10, 4, 3, 800.00, TO_DATE('2026-03-16', 'YYYY-MM-DD'));

```

DONOR table

These insert statements load donor contact records. Each donor has a unique donor_ID, email and phone number so that donation history can be linked back to the correct supporter.

SQL statement:

```

-- DONOR entries
-- -----
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (1,
'Marlene', 'Botha', 'marlene.botha@gmail.com', '0821112233');
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (2,
'Johan', 'Meyer', 'johan.meyer@gmail.com', '0832223344');
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (3,
'Ayesha', 'Naidoo', 'ayesha.naidoo@gmail.com', '0843334455');
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (4,
'Pieter', 'Smit', 'pieter.smit@gmail.com', '0854445566');
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (5,
'Lerato', 'Mokoena', 'lerato.mokoena@gmail.com', '0865556677');
INSERT INTO DONOR (donor_ID, name, surname, email, phone_number) VALUES (6,
'Henry', 'Jacobs', 'henry.jacobs@gmail.com', '0876667788');

```

KENNEL table

These insert statements load the available kennel records used by the shelter. The values include occupied kennels, available kennels and one kennel under maintenance so that the system can demonstrate kennel occupancy and empty-kennel reporting.

SQL statement:

```

-- KENNEL entries
-- -----
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (101,
'Large', 'Occupied');
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (102,
'Small', 'Occupied');
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (103,
'Large', 'Occupied');
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (104,
'Medium', 'Occupied');

```



```
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (105,
'Small', 'Occupied');
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (106,
'Medium', 'Available');
INSERT INTO KENNEL (kennel_ID, sizes, availability_status) VALUES (107,
'Small', 'Maintenance');
```

MEDICAL_PROVISION table

These insert statements load medication and medical supply records. The inStock and cost fields support inventory control and reporting on supplies used during treatments.

SQL statement:

```
-- MEDICAL_PROVISION entries
-- -----
INSERT INTO MEDICAL_PROVISION (medicalProv_ID, description, inStock, cost)
VALUES (1, 'Amoxicillin Tablets', 20, 5.50);
INSERT INTO MEDICAL_PROVISION (medicalProv_ID, description, inStock, cost)
VALUES (2, 'Rabies Vaccine', 1, 85.00);
INSERT INTO MEDICAL_PROVISION (medicalProv_ID, description, inStock, cost)
VALUES (3, 'Sterile Bandage Rolls', 4, 12.00);
```

STAFF table

These insert statements load the general staff records. The STAFF table stores shared employee information such as names, contact details, hire dates and years worked.

SQL statement:

```
-- STAFF entries
-- -----
INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (201, 1, 'Elise', 'Pretorius', 'elise.pretorius@pawsrescue.org',
'0711112222', TO_DATE('2021-02-15', 'YYYY-MM-DD'), 5);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (202, 1, 'Michael', 'Nkosi', 'michael.nkosi@pawsrescue.org',
'0712223333', TO_DATE('2023-05-20', 'YYYY-MM-DD'), 3);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (203, 1, 'Janine', 'Coetzee', 'janine.coetzee@pawsrescue.org',
'0713334444', TO_DATE('2024-01-10', 'YYYY-MM-DD'), 2);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (301, 2, 'Bianca', 'Fourie', 'bianca.fourie@pawsrescue.org',
'0721112222', TO_DATE('2022-03-12', 'YYYY-MM-DD'), 4);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (302, 2, 'Thabo', 'Mokoena', 'thabo.mokoena@pawsrescue.org',
'0722223333', TO_DATE('2023-07-18', 'YYYY-MM-DD'), 3);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
```

```
VALUES (401, 3, 'Anika', 'Venter', 'anika.venter@pawsrescue.org',
'0731112222', TO_DATE('2020-09-01', 'YYYY-MM-DD'), 6);

INSERT INTO STAFF (staff_ID, role_ID, first_name, last_name, email_address,
phone_number, date_hired, years_worked)
VALUES (402, 3, 'Sipho', 'Dlamini', 'sipho.dlamini@pawsrescue.org',
'0732223333', TO_DATE('2022-11-25', 'YYYY-MM-DD'), 4);
```

STAFF_ROLE table

These insert statements load the main staff role categories used in the rescue organisation. The roles are used to classify staff members as veterinarians, veterinary assistants or adoption coordinators.

SQL statement:

```
-- STAFF_ROLE entries
-- -----
INSERT INTO STAFF_ROLE (role_ID, role_name, job_description) VALUES (1,
'Veterinarian', 'Records medical examinations and performs treatments.');
```

```
INSERT INTO STAFF_ROLE (role_ID, role_name, job_description) VALUES (2,
'Veterinary Assistant', 'Assists veterinarians during treatments and daily
animal care.');
```

```
INSERT INTO STAFF_ROLE (role_ID, role_name, job_description) VALUES (3,
'Adoption Coordinator', 'Processes adoption applications and completed
adoptions.');
```

TREATMENT table

These insert statements load treatment types that can be performed on animals. They describe the treatment name, purpose, treatment category and medical instructions.

SQL statement:

```
-- TREATMENT entries
-- -----
INSERT INTO TREATMENT (treatment_ID, name, description, category,
medicalInstruction)
VALUES (401, 'Antibiotic Course', 'Full course of antibiotics for bacterial
infection', 'Medicinal', 'Administer twice daily with food for 7 days');
```

```
INSERT INTO TREATMENT (treatment_ID, name, description, category,
medicalInstruction)
VALUES (402, 'Rabies Vaccination', 'Standard annual rabies vaccination',
'Preventative', 'Single injection, repeat annually');
```

```
INSERT INTO TREATMENT (treatment_ID, name, description, category,
medicalInstruction)
VALUES (403, 'Wound Dressing', 'Cleaning and dressing of open wounds',
'Surgical', 'Clean wound with saline, apply bandage, change every 48 hours');
```

TREATMENT_PROVISION table

These insert statements load the bridge records between treatments and medical provisions. They show which provision is used for each treatment and the quantity used.

SQL statement:

```
-- TREATMENT_PROVISION entries
-- -----
INSERT INTO TREATMENT_PROVISION (treatment_ID, medicalProv_ID, quantity_used)
VALUES (401, 1, 14);
INSERT INTO TREATMENT_PROVISION (treatment_ID, medicalProv_ID, quantity_used)
VALUES (402, 2, 1);
INSERT INTO TREATMENT_PROVISION (treatment_ID, medicalProv_ID, quantity_used)
VALUES (403, 3, 3);
```

TREATMENT_RECORD table

These insert statements load the actual treatment history of animals. Each record links the animal, treatment and staff member and stores the treatment date and result.

SQL statement:

```
-- TREATMENT_RECORD entries
-- -----
INSERT INTO TREATMENT_RECORD (record_ID, animal_ID, treatment_ID, staff_ID,
treatment_date, treatment_result)
VALUES (1, 1, 402, 201, TO_DATE('2026-04-12', 'YYYY-MM-DD'), 'Vaccination
Successful');

INSERT INTO TREATMENT_RECORD (record_ID, animal_ID, treatment_ID, staff_ID,
treatment_date, treatment_result)
VALUES (4, 1, 401, 201, TO_DATE('2026-04-15', 'YYYY-MM-DD'), 'Follow-up
Treatment');

INSERT INTO TREATMENT_RECORD (record_ID, animal_ID, treatment_ID, staff_ID,
treatment_date, treatment_result)
VALUES (2, 2, 403, 202, TO_DATE('2026-04-20', 'YYYY-MM-DD'), 'Wound Treated');

INSERT INTO TREATMENT_RECORD (record_ID, animal_ID, treatment_ID, staff_ID,
treatment_date, treatment_result)
VALUES (3, 3, 401, 203, TO_DATE('2026-03-12', 'YYYY-MM-DD'), 'Antibiotic
Course Started');
```

VETERINARIAN table

These insert statements load veterinarian subtype records. Each veterinarian is linked to a STAFF record and includes a license number, specialization and total animals treated.

SQL statement:

```
-- VETERINARIAN entries
-- -----
INSERT INTO VETERINARIAN (staff_ID, vetAssistant_ID, license_number,
specialization, total_animals_treated)
VALUES (201, 301, 'LIC1001', 'General Surgery', 15);

INSERT INTO VETERINARIAN (staff_ID, vetAssistant_ID, license_number,
specialization, total_animals_treated)
VALUES (202, 302, 'LIC1002', 'Dermatology', 10);

INSERT INTO VETERINARIAN (staff_ID, vetAssistant_ID, license_number,
specialization, total_animals_treated)
VALUES (203, 301, 'LIC1003', 'Spinal Surgery', 12);
```

VETERINARY_ASSISTANT table

These insert statements load veterinary assistant subtype records. Each record is linked to a STAFF record and stores the assistant training status.

SQL statement:

```
-- VETERINARY_ASSISTANT entries
-- -----
INSERT INTO VETERINARY_ASSISTANT (staff_ID, training_status) VALUES (301,
'Completed');
INSERT INTO VETERINARY_ASSISTANT (staff_ID, training_status) VALUES (302, 'In
Progress');
```

Commit Statement

The COMMIT statement saves all inserted rows permanently in the database.

SQL statement:

```
COMMIT;
```

Queries

The following table describes where certain criteria can be found regarding the SQL queries.

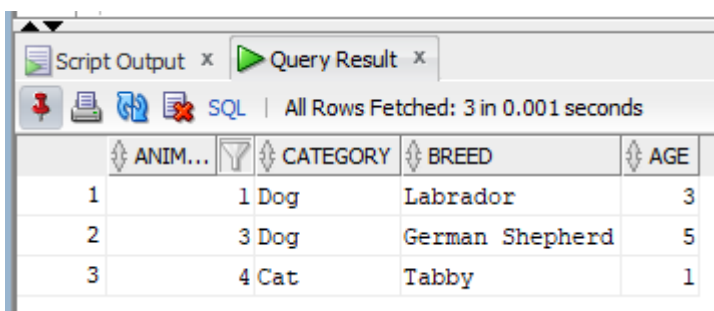
Criterion	Location
Limitations of rows	Statement 2.1, 2.8
Limitation of columns	Statement 2.1, 2.2
Sorting	Statement 2.2, 2.17, 2.18
LIKE	Statement 2.3
AND	Statement 2.3
OR	Statement 2.4
Variables / Functions	Statement 2.6
Character functions	Statement 2.5, 2.6
Round	Statement 2.5, 2.18
Date functions	Statement 2.8, 2.9
Aggregate functions	Statement 2.5, 2.6
Group By	Statement 2.10, 2.11
Having	Statement 2.12
Join	Statement 2.13, 2.14, 2.17
Sub-queries	Statement 2.15, 2.16
Extra functionality	Statement 2.19, 2.20, 2.21

2.1 Limitation of rows and columns

This query displays only the animal identification number, type, breed and age of animals that are currently available for adoption. This limitation of information provided is only needed for the adoption staff when consulting a client regarding their pet.

SQL statement:

```
-- [Limit Columns & Rows] Animals available for adoption
SELECT animal_ID, category, breed, age
FROM ANIMAL
WHERE adoption_availability = 'Available';
```



	ANIM...	CATEGORY	BREED	AGE
1		1 Dog	Labrador	3
2		3 Dog	German Shepherd	5
3		4 Cat	Tabby	1

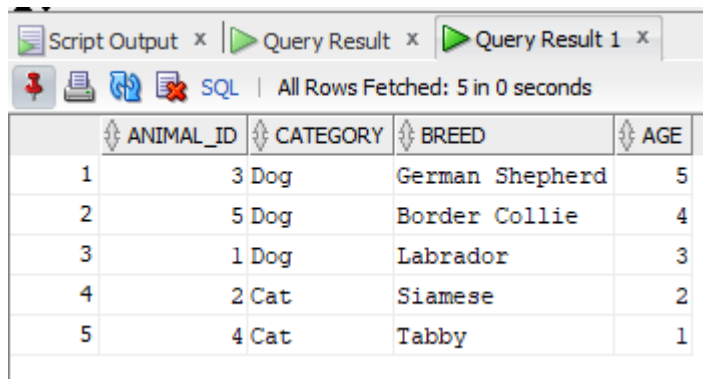
Figure 1 Output of Limitation of rows and columns

2.2 Sorting

This query sorts all animals from oldest to youngest using the ORDER BY clause in descending order. The older animals generally require quicker medical monitoring and can sometimes be prioritised during special adoption campaigns.

SQL statement:

```
-- [Sorting] Animals sorted from oldest to youngest
SELECT animal_ID, category, breed, age
FROM ANIMAL
ORDER BY age DESC;
```



	ANIMAL_ID	CATEGORY	BREED	AGE
1	3	Dog	German Shepherd	5
2	5	Dog	Border Collie	4
3	1	Dog	Labrador	3
4	2	Cat	Siamese	2
5	4	Cat	Tabby	1

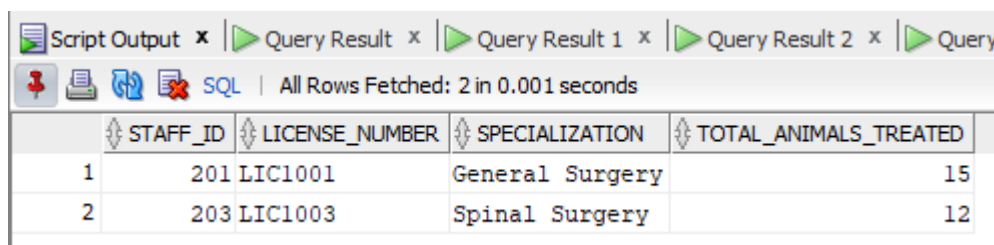
Figure 2 Output of Sorting

2.3 LIKE, AND & OR

This query identifies veterinarians whose specialization contains the word “Surgery” and have treated more than ten animals. The OR query displays animals that are either in critical condition or currently recovering.

SQL statement:

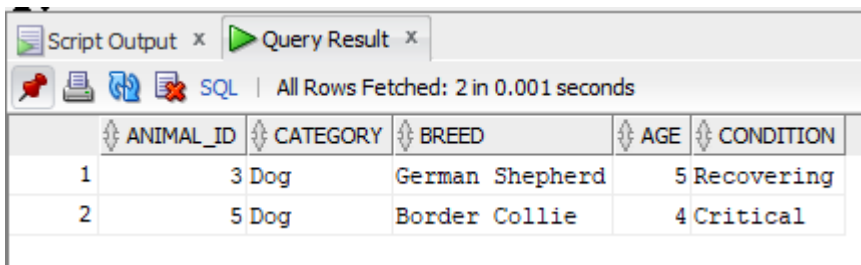
```
-- [LIKE & AND] Vets specialising in Surgery who treated more than 10 animals.
SELECT staff_ID, license_number, specialization, total_animals_treated
FROM VETERINARIAN
WHERE specialization LIKE '%Surgery%'
AND total_animals_treated > 10;
```



	STAFF_ID	LICENSE_NUMBER	SPECIALIZATION	TOTAL_ANIMALS_TREATED
1	201	LIC1001	General Surgery	15
2	203	LIC1003	Spinal Surgery	12

```
-- [OR] Animals in critical or recovering condition
SELECT animal_ID, category, breed, age, condition
FROM ANIMAL
```

```
WHERE condition = 'Critical'
OR condition = 'Recovering';
```



Script Output x Query Result x

SQL | All Rows Fetched: 2 in 0.001 seconds

	ANIMAL_ID	CATEGORY	BREED	AGE	CONDITION
1	3	Dog	German Shepherd	5	Recovering
2	5	Dog	Border Collie	4	Critical

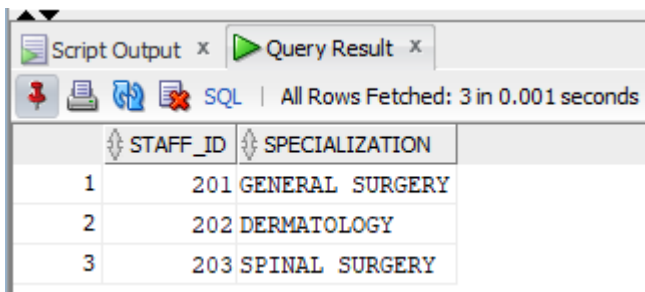
Figure 3 Output of LIKE, AND & OR

2.4 Character functions

This query uses the UPPER character function to convert veterinarian specialization descriptions into uppercase format. This creates a standard display format for the reports and staff identification labels.

SQL statement:

```
-- [Character Functions] Vet specializations in uppercase
SELECT staff_ID,
       UPPER(specialization) AS SPECIALIZATION
FROM VETERINARIAN;
```



Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.001 seconds

	STAFF_ID	SPECIALIZATION
1	201	GENERAL SURGERY
2	202	DERMATOLOGY
3	203	SPINAL SURGERY

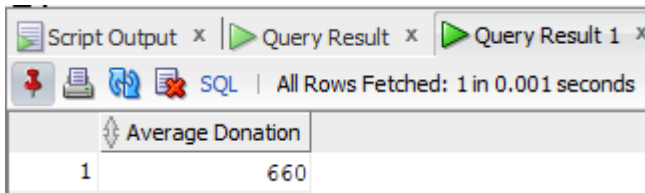
Figure 4 Output of Variables and character functions

2.5 Round

This query calculates the average donation amount and rounds the result to two decimal places. It helps management understand typical donation values in a clear and readable format.

SQL statement:

```
-- [ROUND] Average donation amount rounded to 2 decimal places
SELECT ROUND(AVG(donation_amount), 2) AS "Average Donation"
FROM DONATION;
```



Average Donation	
1	660

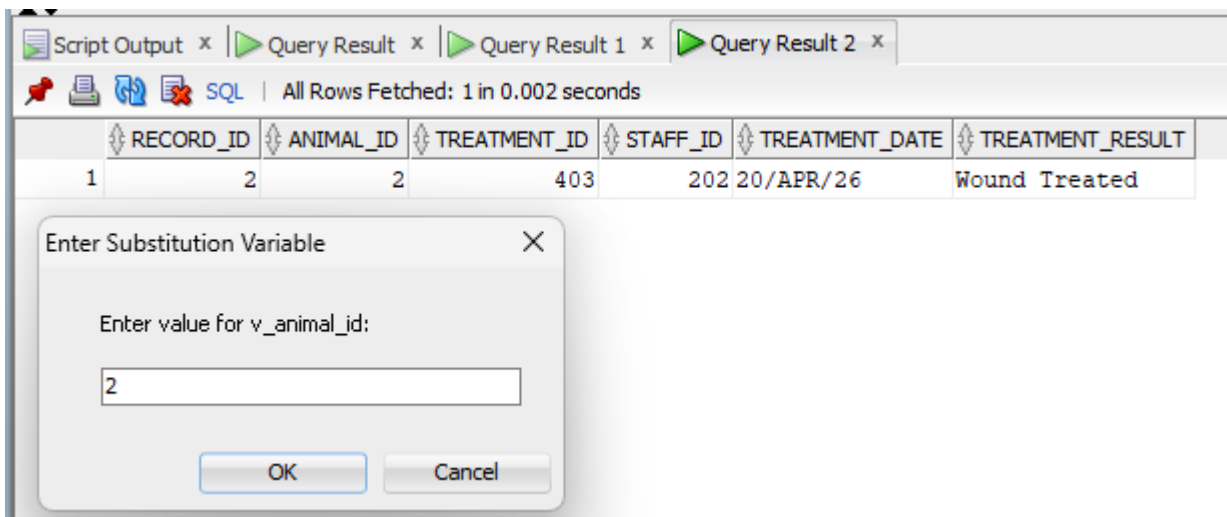
Figure 5 Output of Round and/or trunc

2.6 Variables

This query is used to retrieve treatment records for a specific animal by allowing the user to input an animal ID at runtime. It is useful when searching for detailed medical history of a particular animal.

SQL statement:

```
-- [Variables & Character Functions] Prompt for an animal ID; show treatment
records
SELECT * FROM TREATMENT_RECORD
WHERE animal_ID = &v_animal_id;
```



RECORD_ID	ANIMAL_ID	TREATMENT_ID	STAFF_ID	TREATMENT_DATE	TREATMENT_RESULT
1	2	403	202	20/APR/26	Wound Treated

Enter Substitution Variable

Enter value for v_animal_id:

OK Cancel

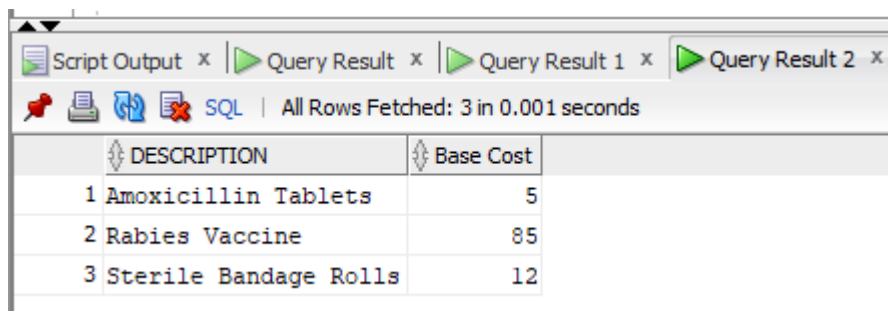
Figure 6 Output of Data functions

2.7 Trunc

This query removes the decimal portion of medical provision costs, showing only the base cost. It is useful for simplified reporting and cost comparisons.

SQL statement:

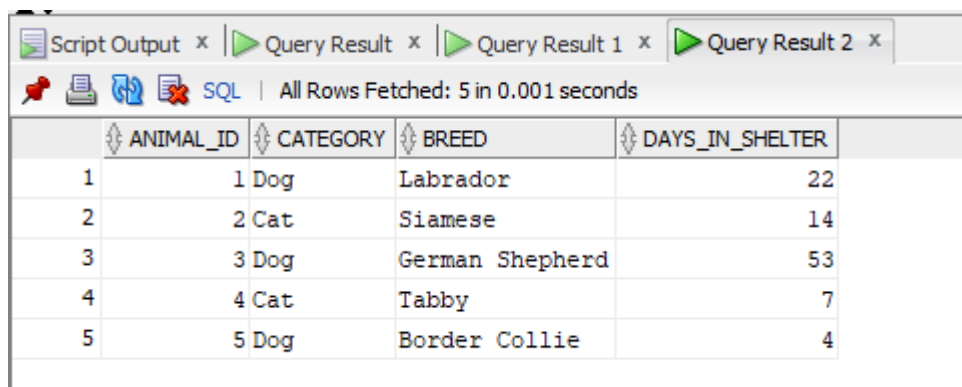
```
-- [TRUNC] Base cost of each medical provision
SELECT description, TRUNC(cost) AS "Base Cost"
FROM MEDICAL_PROVISION;
```



The screenshot shows a SQL query result window with the following data:

	DESCRIPTION	Base Cost
1	Amoxicillin Tablets	5
2	Rabies Vaccine	85
3	Sterile Bandage Rolls	12

```
-- [TRUNC / Date] Days each animal has been in the shelter
SELECT animal_ID, category, breed,
       TRUNC(SYSDATE - arrivalDate) AS days_in_shelter
FROM ANIMAL;
```



The screenshot shows a SQL query result window with the following data:

	ANIMAL_ID	CATEGORY	BREED	DAYS_IN_SHELTER
1	1	Dog	Labrador	22
2	2	Cat	Siamese	14
3	3	Dog	German Shepherd	53
4	4	Cat	Tabby	7
5	5	Dog	Border Collie	4

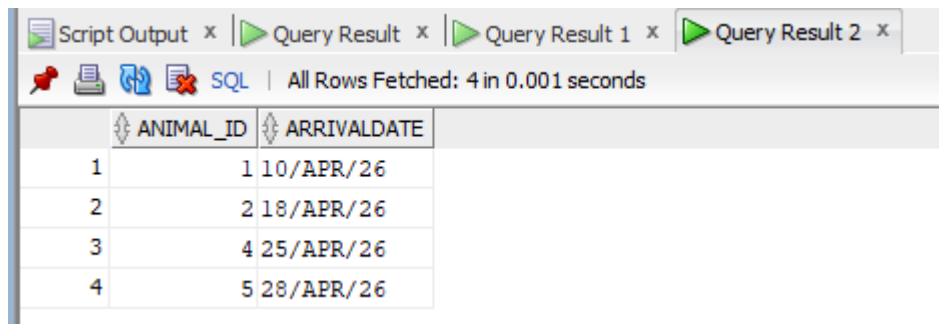
Figure 7 Output of Group by and having

2.8 Date

This query retrieves animals that have arrived at the rescue within the last 30 days. It helps staff monitor recent intakes and manage new arrivals.

SQL statement:

```
-- [Date Functions] Animals that arrived in the last 30 days 2.8
SELECT animal_ID, arrivalDate
FROM ANIMAL
WHERE arrivalDate >= SYSDATE - 30;
```



The screenshot shows a SQL query result window with the title 'Query Result 2'. It displays a table with two columns: 'ANIMAL_ID' and 'ARRIVALDATE'. The table contains four rows of data. The status bar at the top indicates 'All Rows Fetched: 4 in 0.001 seconds'.

	ANIMAL_ID	ARRIVALDATE
1	1	10/APR/26
2	2	18/APR/26
3	4	25/APR/26
4	5	28/APR/26

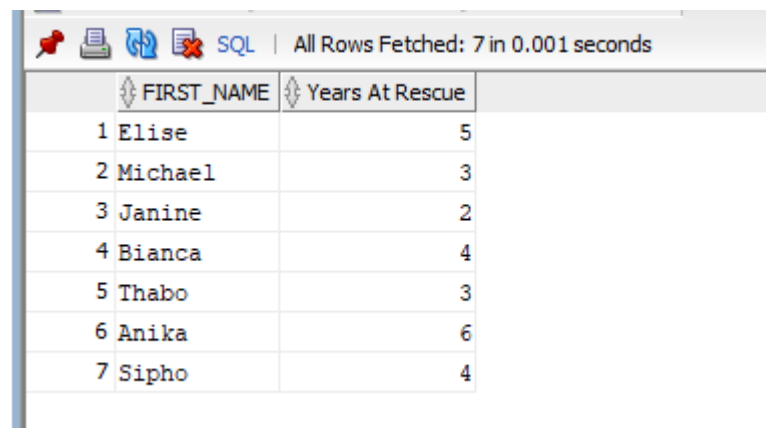
Figure 8 Output of Date

2.9 Date Function

This query calculates how many years each staff member has worked at the rescue. It is useful for tracking staff experience and tenure.

SQL statement:

```
-- [Date Functions] Years each staff member has been employed
SELECT first_name,
       EXTRACT(YEAR FROM SYSDATE) - EXTRACT(YEAR FROM date_hired) AS "Years At
Rescue"
FROM STAFF;
```



The screenshot shows a SQL query result window with the title 'Query Result 1'. It displays a table with two columns: 'FIRST_NAME' and 'Years At Rescue'. The table contains seven rows of data. The status bar at the top indicates 'All Rows Fetched: 7 in 0.001 seconds'.

	FIRST_NAME	Years At Rescue
1	Elise	5
2	Michael	3
3	Janine	2
4	Bianca	4
5	Thabo	3
6	Anika	6
7	Sipho	4

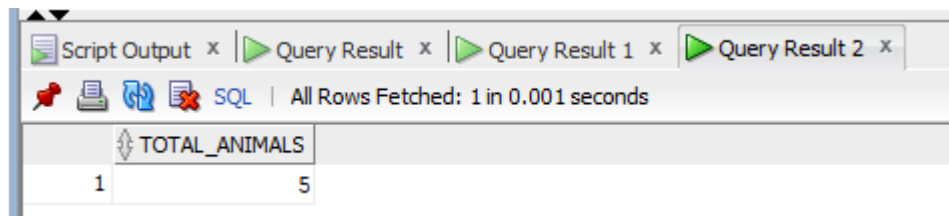
Figure 9 Output of Date Function

2.10 Aggregates

This query counts the total number of animals currently in the shelter. It uses the COUNT aggregation function on animals_id to return a single total value.

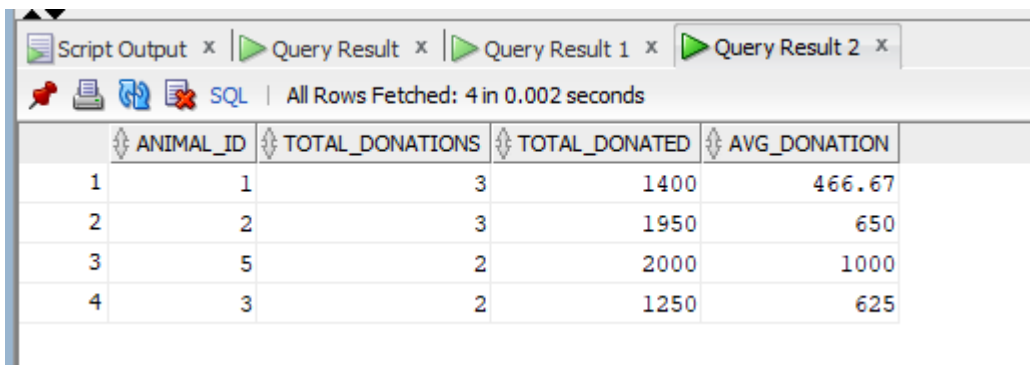
SQL statement:

```
-- [Aggregate Functions] Total current animals in shelter
SELECT COUNT(animals_id) AS Total_Animals
FROM ANIMAL;
```



TOTAL_ANIMALS
5

```
-- [Aggregate Functions] Total and average donation per animal
SELECT animal_ID,
       COUNT(*) AS total_donations,
       SUM(donation_amount) AS total_donated,
       ROUND(AVG(donation_amount), 2) AS avg_donation
FROM DONATION
GROUP BY animal_ID;
```



ANIMAL_ID	TOTAL_DONATIONS	TOTAL_DONATED	AVG_DONATION
1	1	1400	466.67
2	2	1950	650
3	5	2000	1000
4	3	1250	625

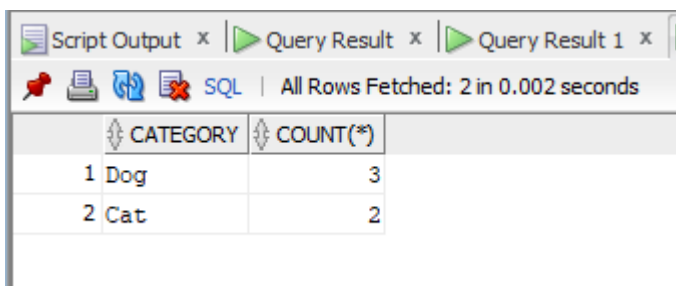
Figure 10 Output of Aggregates

2.11 Group By

This query counts how many animals exist for each type (such as cat, dog, etc.). It groups the records by type and then applies the COUNT function to each group.

SQL statement:

```
-- [Group By] Count how many animals exists in group category (Cat, Dog)
SELECT category, COUNT(*) FROM ANIMAL GROUP BY category;
```



CATEGORY	COUNT(*)
1 Dog	3
2 Cat	2

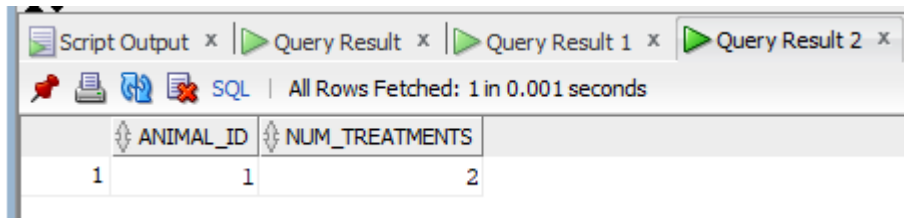
Figure 11 Output of Group By

2.12 Having

This query identifies breeds that have more than 5 animals in the shelter. It groups animal by breed and then filter the grouped results using the HAVING clause.

SQL statement:

```
-- [Group By & Having] Animals that have received more than one treatment
SELECT animal_ID, COUNT(*) AS num_treatments
FROM TREATMENT_RECORD
GROUP BY animal_ID
HAVING COUNT(*) > 1;
```



ANIMAL_ID	NUM_TREATMENTS
1	2

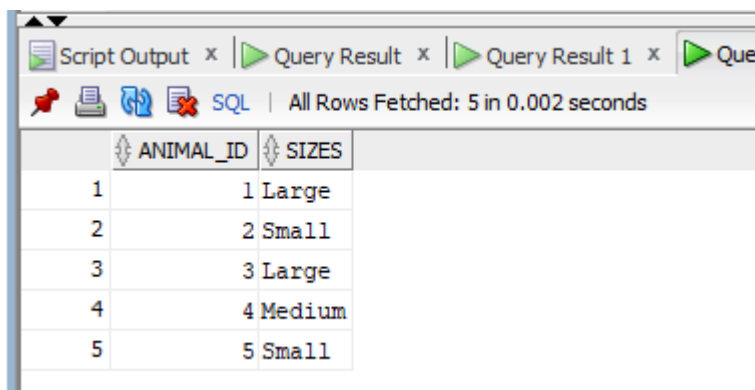
Figure 12 Output of Having

2.13 Join (Inner)

This query lists animals together with the size of the kennel they are assigned to. It uses INNER JOIN between ANIMAL and KENNEL based on kennel_id to match related records

SQL statement:

```
-- [Inner Join] lists animals together with the sizes of the kennel they are
assigned to
SELECT A.animal_id, K.sizes
FROM ANIMAL A
JOIN KENNEL K ON A.kennel_id = K.kennel_id;
```



ANIMAL_ID	SIZES
1	1 Large
2	2 Small
3	3 Large
4	4 Medium
5	5 Small

Figure 13 Output of Join (Inner)

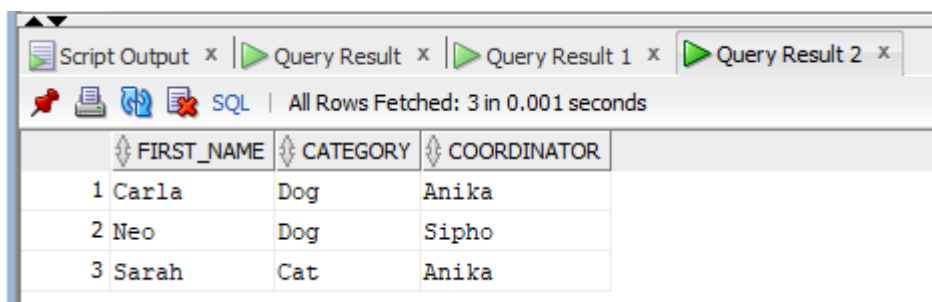
2.14 Join (Multiple)

This query shows which adopter adopts which animal and displays the coordinator (staff member) involved in the adoption process. It joins ADOPTIONS, ADOPTER, ANIMALS, and STAFF tables to combine all related information

SQL statement:

```
-- [Join Multiple] Shows which adopter adopted which animal and displays the coordinator
```

```
SELECT ADO.first_name, AN.category, ST.first_name AS Coordinator
FROM ADOPTION AD
JOIN ADOPTER ADO ON AD.adopter_id = ADO.adopter_id
JOIN ANIMAL AN ON AD.animal_id = AN.animal_id
JOIN STAFF ST ON AD.staff_id = ST.staff_id;
```

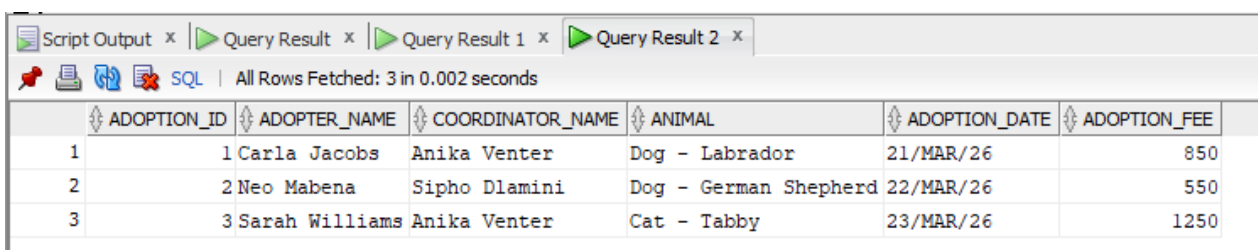


The screenshot shows a SQL query result window with the following data:

	FIRST_NAME	CATEGORY	COORDINATOR
1	Carla	Dog	Anika
2	Neo	Dog	Sipho
3	Sarah	Cat	Anika

```
-- [Joins] Full adoption details which include adopter, coordinator, animal, fee
```

```
SELECT a.adoption_ID,
       ad.first_name || ' ' || ad.last_name AS adopter_name,
       s.first_name || ' ' || s.last_name AS coordinator_name,
       an.category || ' - ' || an.breed AS animal,
       a.adoption_date,
       a.adoption_fee
FROM ADOPTION a
JOIN ADOPTER ad ON a.adopter_ID = ad.adopter_ID
JOIN ADOPTION_COORDINATOR ac ON a.staff_ID = ac.staff_ID
JOIN STAFF s ON ac.staff_ID = s.staff_ID
JOIN ANIMAL an ON a.animal_ID = an.animal_ID;
```



The screenshot shows a SQL query result window with the following data:

	ADOPTION_ID	ADOPTER_NAME	COORDINATOR_NAME	ANIMAL	ADOPTION_DATE	ADOPTION_FEE
1	1	1 Carla Jacobs	Anika Venter	Dog - Labrador	21/MAR/26	850
2	2	2 Neo Mabena	Sipho Dlamini	Dog - German Shepherd	22/MAR/26	550
3	3	3 Sarah Williams	Anika Venter	Cat - Tabby	23/MAR/26	1250

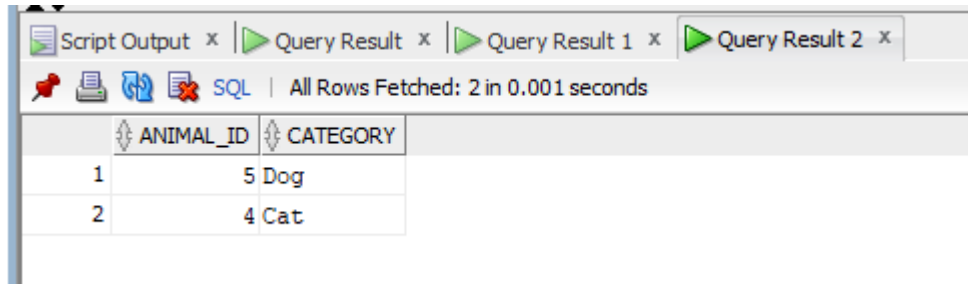
Figure 14 Output of Join (Multiple)

2.15 Sub-query

This query finds animals that have never received any treatment. It uses a sub-query to check which animal_ID values appear in TREATMENT_RECORD and then excludes those animals from the result.

SQL statement:

```
-- [Sub-queries] Animals that have not received any treatments
SELECT animal_ID, category
FROM ANIMAL
WHERE animal_ID NOT IN (
    SELECT animal_ID FROM TREATMENT_RECORD
);
```



ANIMAL_ID	CATEGORY
1	5 Dog
2	4 Cat

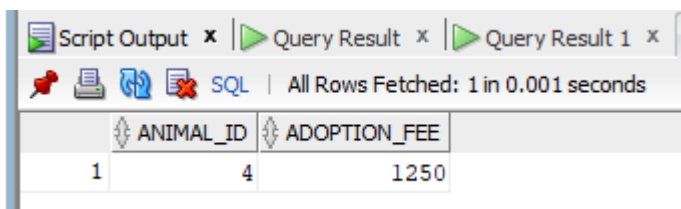
Figure 15 Output of Sub-query

2.16 Sub-query

This query lists adoption records where the adoption fee is higher than the average adoption fee. It uses a sub-query to calculate the average adoption fee and compares every adoption fee against that value.

SQL statement:

```
-- [Sub-queries] Adoptions with a fee above the average adoption fee
SELECT animal_ID, adoption_fee
FROM ADOPTION
WHERE adoption_fee > (
    SELECT AVG(adoption_fee) FROM ADOPTION
);
```



ANIMAL_ID	ADOPTION_FEE
1	1250

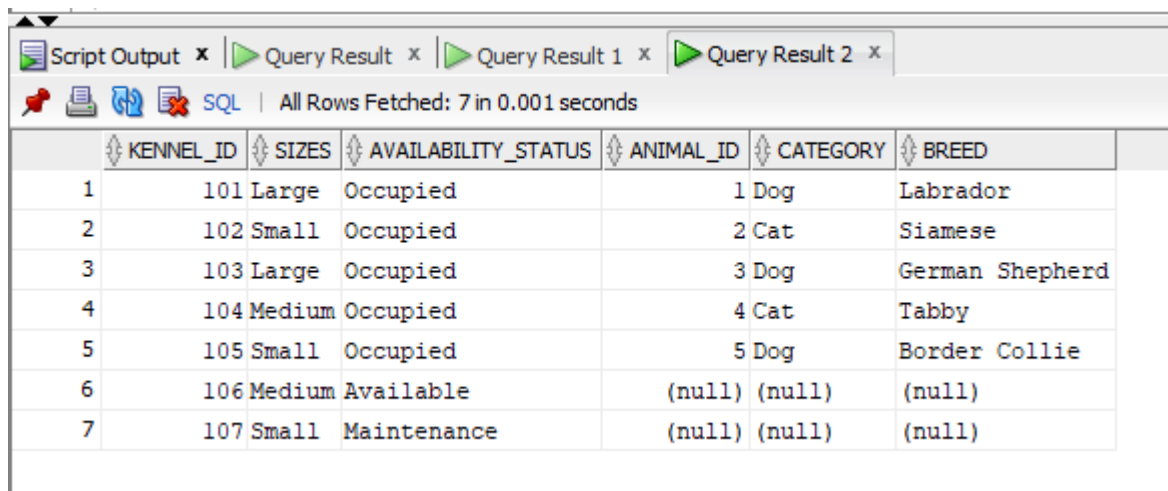
Figure 16 Output of Sub-query

2.17 Outer Join

This query lists all kennels and the animals assigned to them. A LEFT JOIN is used so that kennels without animals still appear in the output, which helps management identify empty kennels.

SQL statement:

```
-- [OUTER Join] All kennels with their assigned animals
SELECT k.kennel_ID, k.sizes, k.availability_status,
       a.animal_ID, a.category, a.breed
FROM KENNEL k
LEFT OUTER JOIN ANIMAL a ON k.kennel_ID = a.kennel_ID
ORDER BY k.kennel_ID;
```



	KENNEL_ID	SIZES	AVAILABILITY_STATUS	ANIMAL_ID	CATEGORY	BREED
1	101	Large	Occupied	1	Dog	Labrador
2	102	Small	Occupied	2	Cat	Siamese
3	103	Large	Occupied	3	Dog	German Shepherd
4	104	Medium	Occupied	4	Cat	Tabby
5	105	Small	Occupied	5	Dog	Border Collie
6	106	Medium	Available	(null)	(null)	(null)
7	107	Small	Maintenance	(null)	(null)	(null)

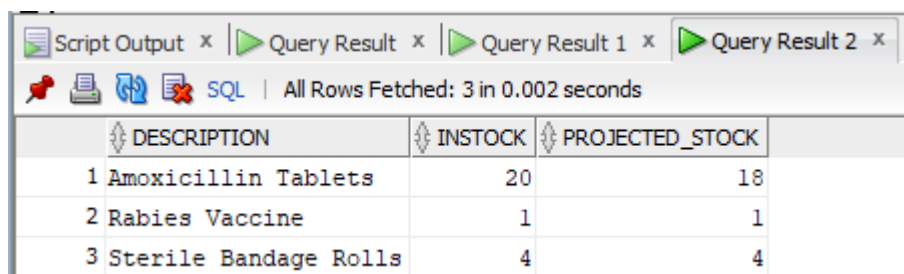
Figure 178 Output of Outer Join

2.18 Math & Logic

This query calculates the projected stock level of each medical provision after a hypothetical 10% usage. It multiplies current stock by 0.9 to show the estimated stock remaining.

SQL statement:

```
-- [Math & Logic] Projected stock after 10% usage
SELECT description,
       inStock,
       ROUND(inStock * 0.9, 0) AS projected_stock
FROM MEDICAL_PROVISION
ORDER BY description;
```



	DESCRIPTION	INSTOCK	PROJECTED_STOCK
1	Amoxicillin Tablets	20	18
2	Rabies Vaccine	1	1
3	Sterile Bandage Rolls	4	4

Figure 19 Output of Math & Logic

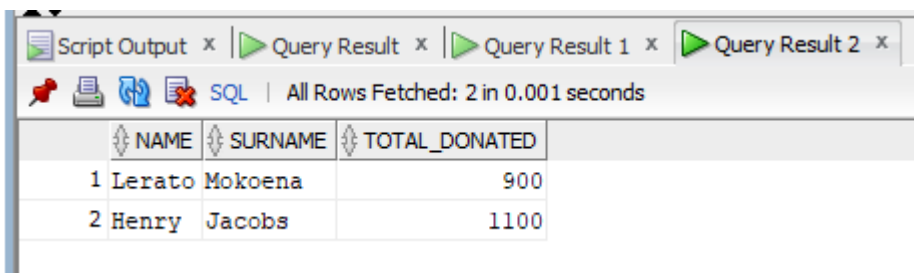
Extra Functionality

2.19 High Value Donors support Critical Animals

This query identifies donors who have given above average donation amounts, specifically for animals in critical condition. It joins the DONOR and DONATION tables and filters for donations which are associated with animals whose condition is “Critical”. The results are then grouped by donor name, surname and the HAVING clause filters out any donors whose total giving amount is less than the overall average donation amount which is calculated using a correlated subquery. This is especially useful for Paws and Claws Rescue to identify and develop relationships with their most impactful supporters and recognise donors who are motivated by urgent animal welfare cases.

SQL statement:

```
-- High-value donors who donated to critical animals above the average
donation total
SELECT D.name, D.surname, SUM(DN.donation_amount) AS Total_Donated
FROM DONOR D
JOIN DONATION DN ON D.donor_id = DN.donor_id
WHERE DN.animal_id IN (SELECT animal_id FROM ANIMAL WHERE condition =
'Critical')
GROUP BY D.name, D.surname
HAVING SUM(DN.donation_amount) > (SELECT AVG(donation_amount) FROM DONATION);
```



	NAME	SURNAME	TOTAL_DONATED
1	Lerato	Mokoena	900
2	Henry	Jacobs	1100

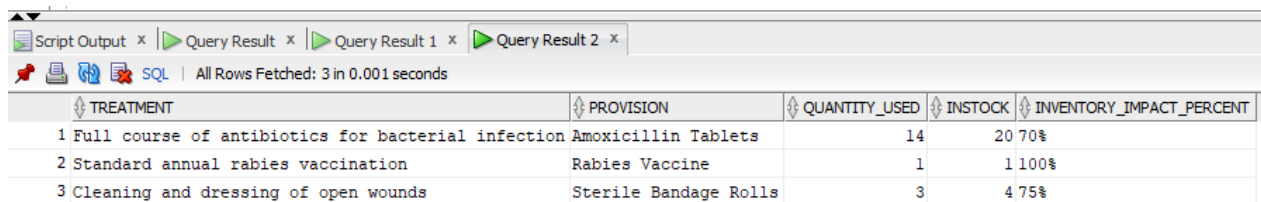
Figure 209 High Value Donors support Critical Animals

2.20 Medical Provision Utilisation Ratio

This query calculates the utilization impact percentage for each medical provision used across treatments. It joins the TREATMENT, TREATMENT_PROVISION and MEDICAL_PROVISION tables to establish what proportion of the current stock of each provision has been consumed during treatments. The NULLIF function is a defensive approach to avoid division by zero errors when a provision's stock is zero. The result is then rounded to two decimal places and is shown as a percentage value. The WHERE clause filters the results to show only provisions where more than 50% of the stock has been consumed, alerting staff that some provisions require urgent restocking. By highlighting which medical supplies are under strain, this query will be a useful tool for inventory management, preventing treatment delays caused by medical provision shortages.

SQL statement:

```
-- Calculates the "Utilization Ratio" of medical provisions.
SELECT T.description AS Treatment,
       MP.description AS Provision,
       TP.quantity_used,
       MP.inStock,
       ROUND((TP.quantity_used / NULLIF(MP.inStock, 0)) * 100, 2) || '%' AS
Inventory_Impact_Percent
FROM TREATMENT T
JOIN TREATMENT_PROVISION TP ON T.treatment_id = TP.treatment_id
JOIN MEDICAL_PROVISION MP ON TP.medicalProv_ID = MP.medicalProv_ID
WHERE (TP.quantity_used / NULLIF(MP.inStock, 0)) > 0.5;
```



The screenshot shows a database query result with the following data:

TREATMENT	PROVISION	QUANTITY_USED	INSTOCK	INVENTORY_IMPACT_PERCENT
1 Full course of antibiotics for bacterial infection	Amoxicillin Tablets	14	20	70%
2 Standard annual rabies vaccination	Rabies Vaccine	1	1	100%
3 Cleaning and dressing of open wounds	Sterile Bandage Rolls	3	4	75%

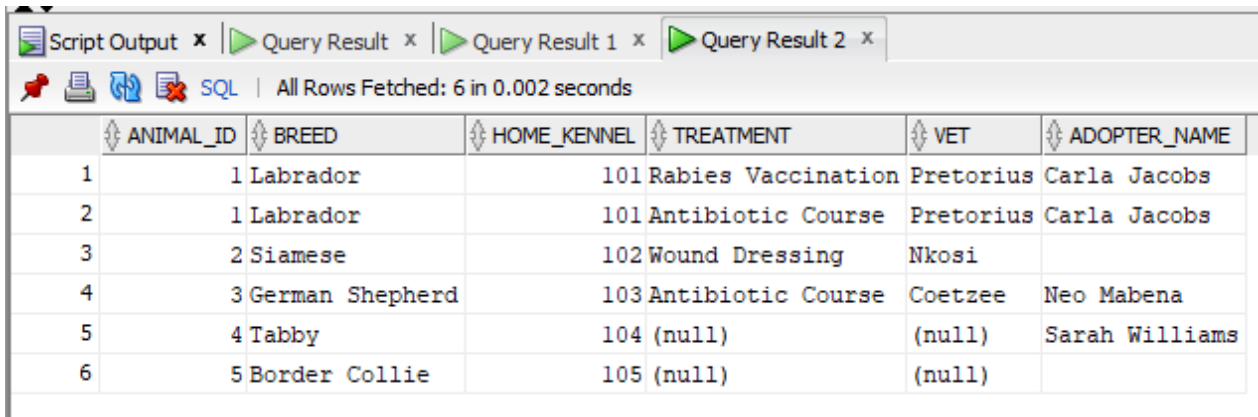
Figure 21 Output of Medical Provision Utilisation Ratio

2.21 Complete Animal History

This query joins five different tables to provide a comprehensive “life at the Paws and Claws Rescue shelter” report for each animal in the database. It retrieves the animal’s ID and breed from the ANIMAL table, the kennel they were in from the KENNEL table, the medical treatments they received align with the veterinarian staff member who gave it to them from the TREATMENT_RECORD, TREATMENT and VETERINARIAN tables and finally the details of the adopter who took them home from the ADOPTION and ADOPTER tables. The LEFT JOIN is used everywhere to ensure that animals have not yet received treatment or not yet been adopted are still include in the results

SQL statement:

```
-- Full animal report which includes kennel, treatment, vet, adopter
SELECT
  a.animal_ID,
  a.breed,
  k.kennel_ID AS home_kennel,
  t.name AS treatment,
  s.last_name AS vet,
  ad.first_name || ' ' || ad.last_name AS adopter_name
FROM ANIMAL a
JOIN KENNEL k ON a.kennel_ID = k.kennel_ID
LEFT JOIN TREATMENT_RECORD tr ON a.animal_ID = tr.animal_ID
LEFT JOIN TREATMENT t ON tr.treatment_ID = t.treatment_ID
LEFT JOIN STAFF s ON tr.staff_ID = s.staff_ID
LEFT JOIN ADOPTION adp ON a.animal_ID = adp.animal_ID
LEFT JOIN ADOPTER ad ON adp.adopter_ID = ad.adopter_ID
ORDER BY a.animal_ID;
```



The screenshot shows a SQL query result window with the following tabs: 'Script Output', 'Query Result', 'Query Result 1', and 'Query Result 2'. The 'Query Result' tab is active, displaying a table with 6 rows and 7 columns. The columns are: ANIMAL_ID, BREED, HOME_KENNEL, TREATMENT, VET, and ADOPTER_NAME. The data is as follows:

	ANIMAL_ID	BREED	HOME_KENNEL	TREATMENT	VET	ADOPTER_NAME
1	1	Labrador	101	Rabies Vaccination	Pretorius	Carla Jacobs
2	1	Labrador	101	Antibiotic Course	Pretorius	Carla Jacobs
3	2	Siamese	102	Wound Dressing	Nkosi	
4	3	German Shepherd	103	Antibiotic Course	Coetzee	Neo Mabena
5	4	Tabby	104	(null)	(null)	Sarah Williams
6	5	Border Collie	105	(null)	(null)	

Figure 2210 Output of Complete Animal History